

Sequence models



Recurrent Neural Networks

Supervised learning

exps:

Input

output

speech recognition

(X)

(Y)

Input



Audio

transcript

music generation

$\emptyset$



genre of
the music
entered



Music
generated
(sequence)

Sentiment classification

"there is nothing
in this movie so
be liked."

→ rating the
movie

sequence



DNA sequence
analysis

"AGCCATTC"



"find out the
specific
part of the
sequence like
exon, intron"

②

machine translation

"wo bin ich?" $\rightarrow$ "where am I?"

video activity recognition



$\rightarrow$ running

name entity recognition

given sentence $\rightarrow$ identify the people in that sentence.

notation

example (~~identify~~ name entity recognition)

X: Harry and Hermione invented a new spell.

Y: 1 0 -1 0 0 0 0 .
 $y_{(1)}$ $y_{(2)}$ $y_{(3)}$ $y_{(4)}$ $y_{(5)}$ $y_{(6)}$ $y_{(7)}$
 indexing one binary bit for each word in the sentence

$$T_x = 9 \rightarrow \text{len}(X)$$

$$T_y = 9 \rightarrow \text{len}(Y)$$

$X^{(i)}_{(t)}$: ~~the~~ t th element of the sequence i in data set X .

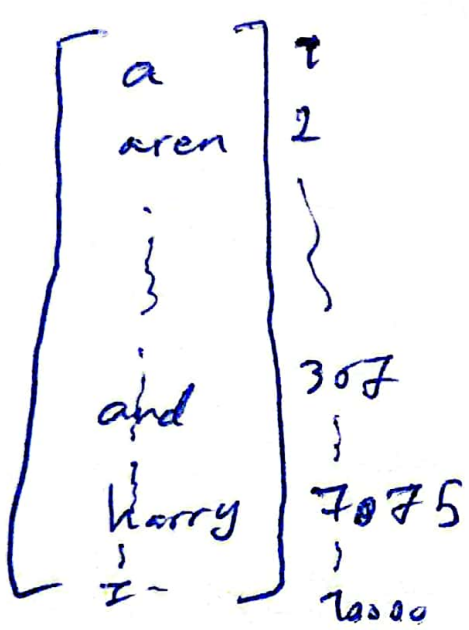
$T_x^{(i)}$ = length of input example of X data set

$$T_y^{(i)} \langle t \rangle = p$$

$$T_y^{(i)} = p$$

representing words;

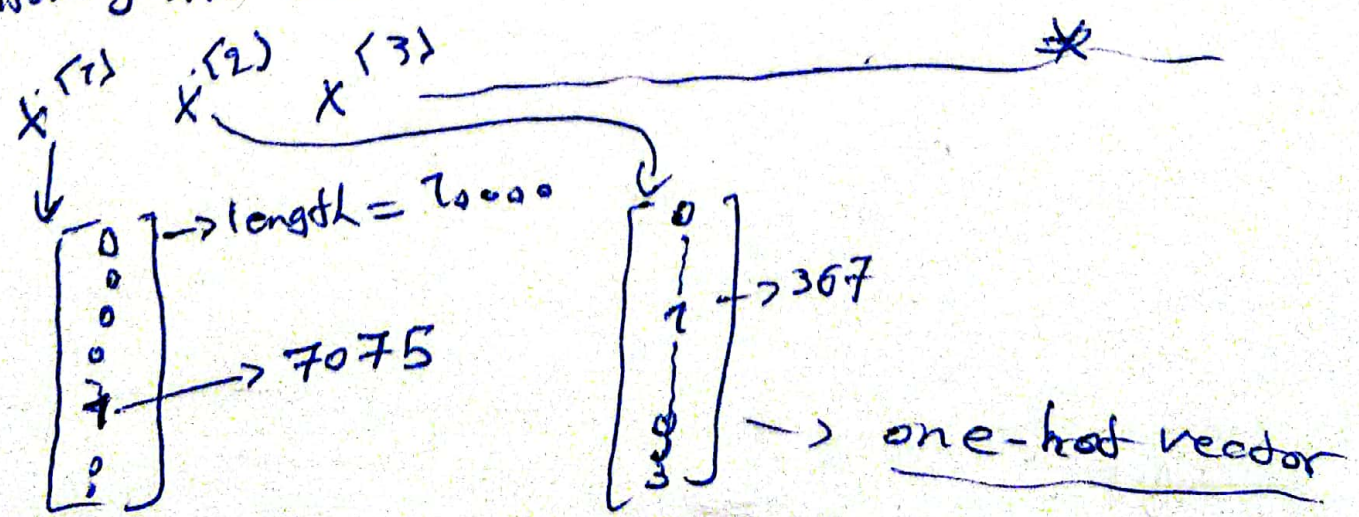
Vocabulary (dictionary)



popular
10000 common words in
english

10000 word
dictionary

Harry and Hermione invented a new spell.



if a word does not exist in the vocabulary, a new token will be created, called unknown words.
~~{UNK}~~
 {UNK}

Recurrent neural network model

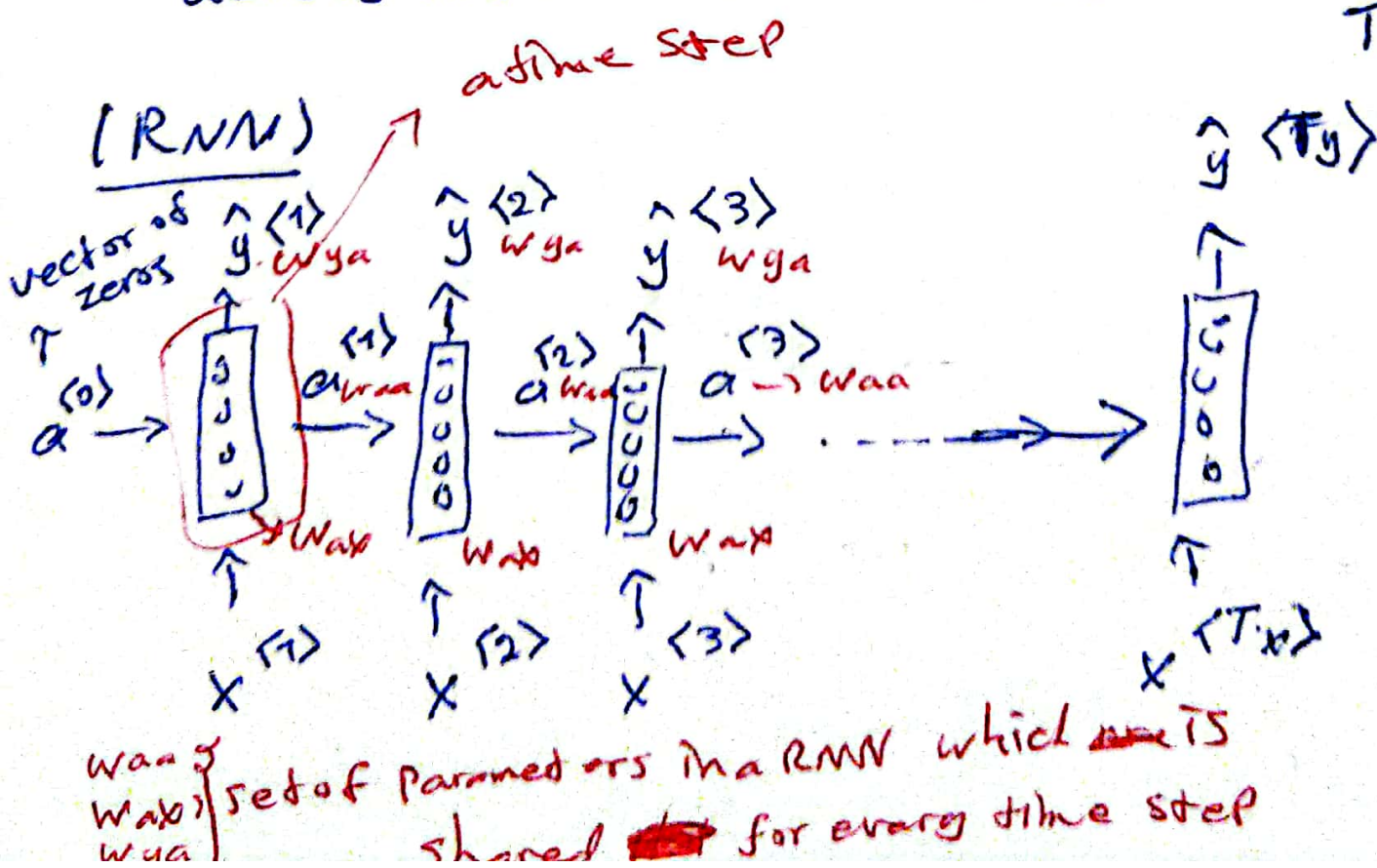
(MLP)

why not a standard network?

problem 1: Input and outputs can be different lengths in different examples.

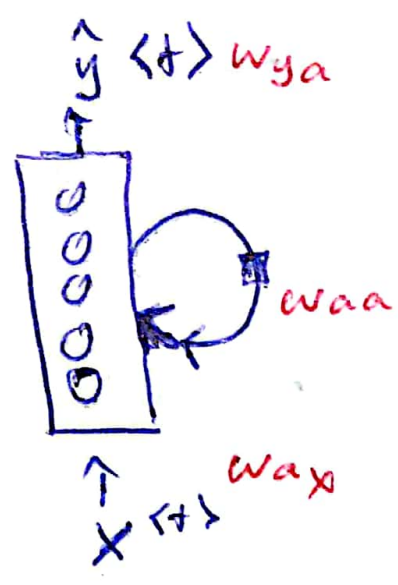
problem 2: Does not share features learned across different positions of text.

$$T_x = T_y$$



5

another representation of an RNN.



→ a unidirectional RNN

activation function

$$a^{(0)} = \vec{0} \quad \left| \quad \begin{aligned} a^{(t)} &= g(w_{aa} a^{(t-1)} + w_{ax} x^{(t)} + b_a) \\ \hat{y}^{(t)} &= g(w_{ya} a^{(t)} + b_y) \end{aligned} \right.$$

activation function:
 ~~small~~ tanh/
 ReLU

 sigmoid
 softmax
 ...

$$a^{(t)} = g(w_{aa} a^{(t-1)} + w_{ax} x^{(t)} + b_a)$$
$$\hat{y}^{(t)} = g(w_{ya} a^{(t)} + b_y)$$

$$a^{(t)} = g(w_a [a^{(t-1)}, x^{(t)}] + b_a)$$

$$w_a = [w_{aa}, w_{ax}]$$

stacked together. (hidden)

5

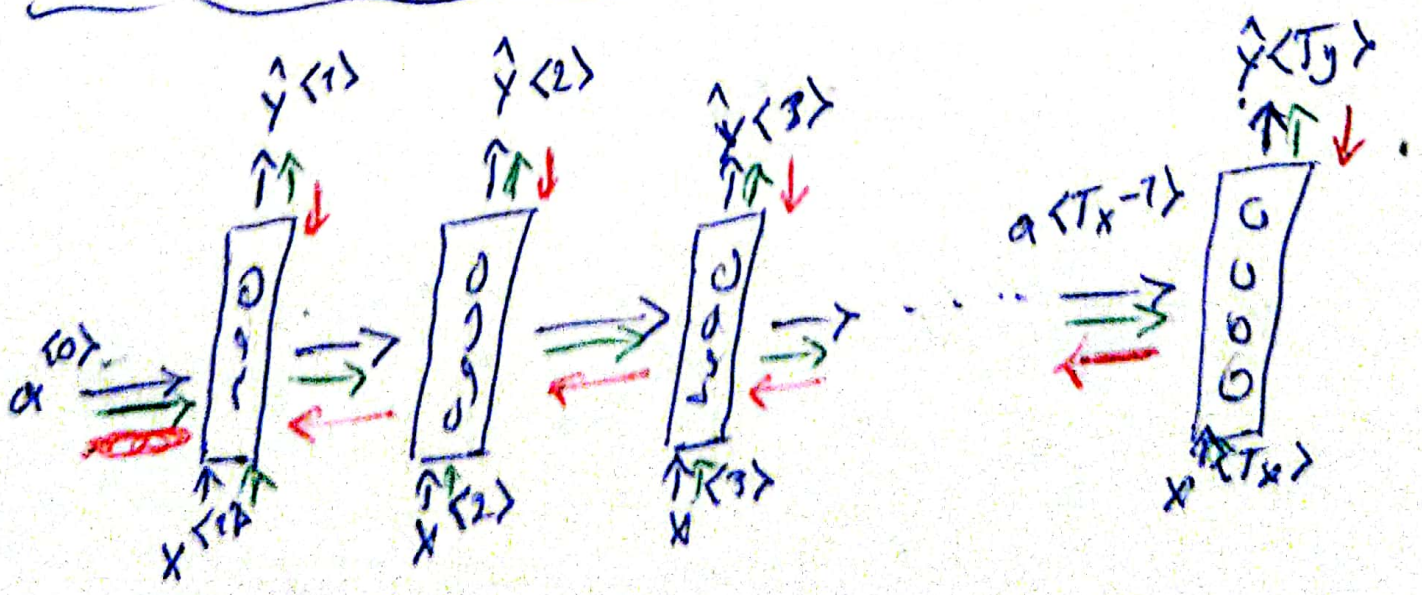
$$[a^{(t-1)}, x^{(t)}] = \begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} \rightarrow \text{stacked together (Vstack)}$$

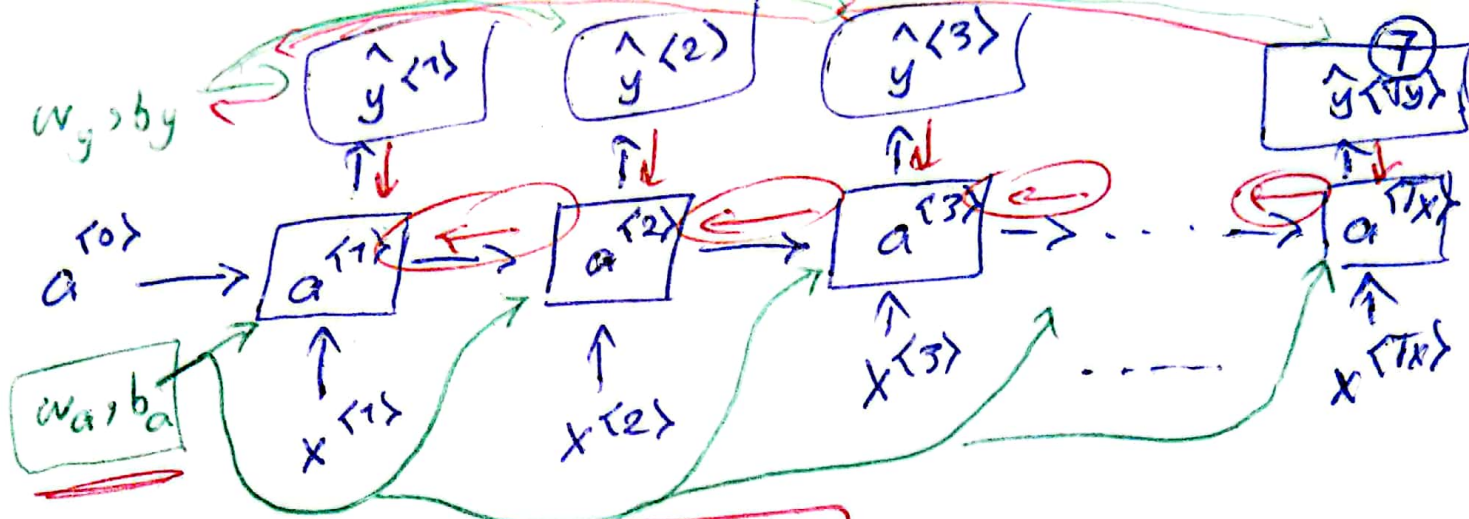
$$[w_{aa}, w_{ax}] \begin{bmatrix} a^{(t-1)} \\ x^{(t)} \end{bmatrix} = w_{aa} a^{(t-1)} + w_{ax} x^{(t)}$$

$$\hat{y}^{(t)} = g(w_{ya} a^{(t)} + b_y)$$

$$\hat{y}^{(t)} = g(\underbrace{w_{ya}}_{\text{Params}} a^{(t)} + \underbrace{b_y}_{\text{Params}})$$

Back propagation through time: backward forward





back propagation through time

$$L^{(t)}(\hat{y}^{(t)}, y^{(t)}) = -y^{(t)} \log \hat{y}^{(t)} - (1 - y^{(t)}) \log (1 - \hat{y}^{(t)})$$

log loss

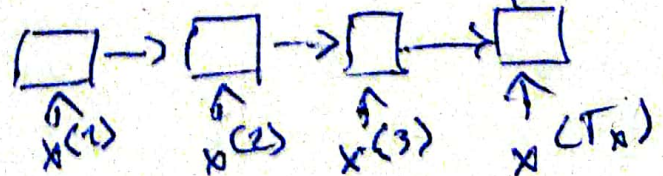
$$L(\hat{y}|y) = \sum_{t=1}^{T_y} L^{(t)}(\hat{y}^{(t)}, y^{(t)})$$

loss for the entire sequence

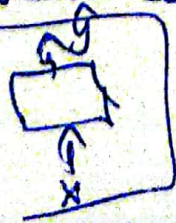
different types of RNNs

- many to many architectures : $T_x = T_y$ → one output

- many-to-one architectures



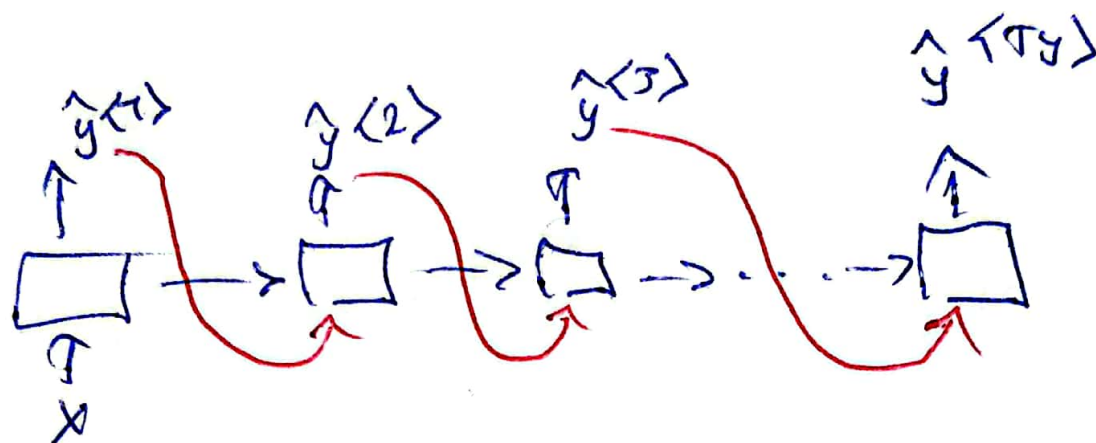
- one-to-one



- one-to-many (music generation)

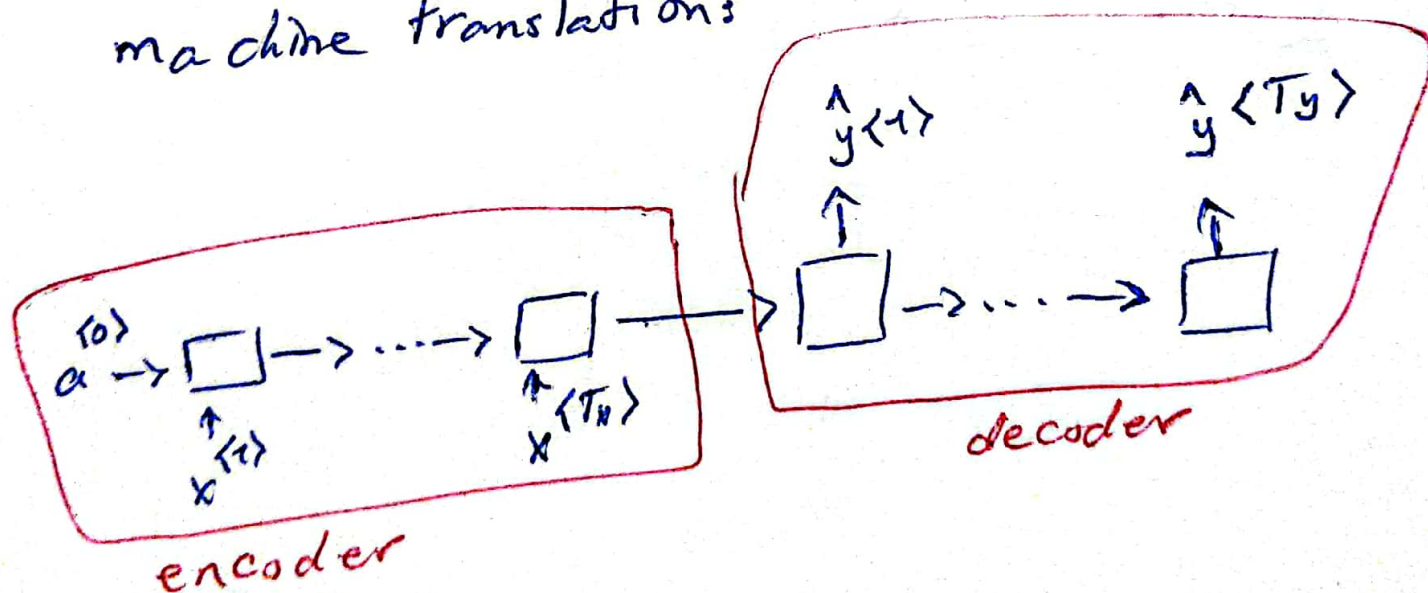
⑧

$$x \rightarrow y^{(1)} y^{(2)} \dots y^{(T_y)}$$

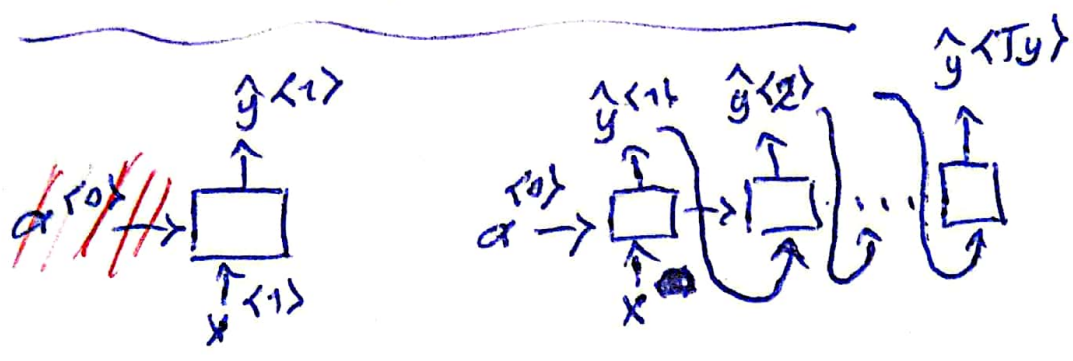


exp, (many to many):

machine translation:

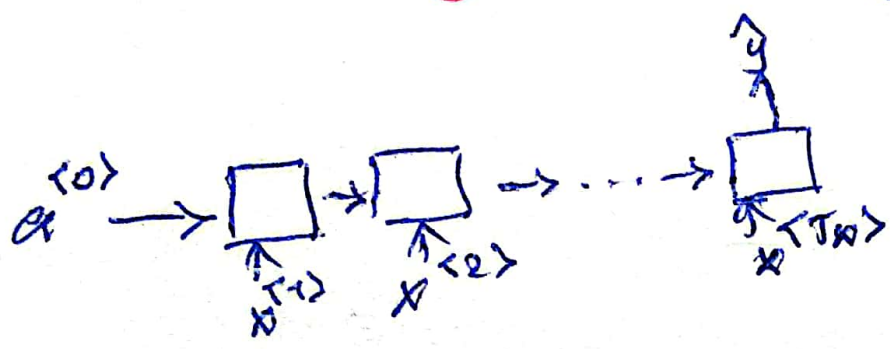


Summary of RNN types

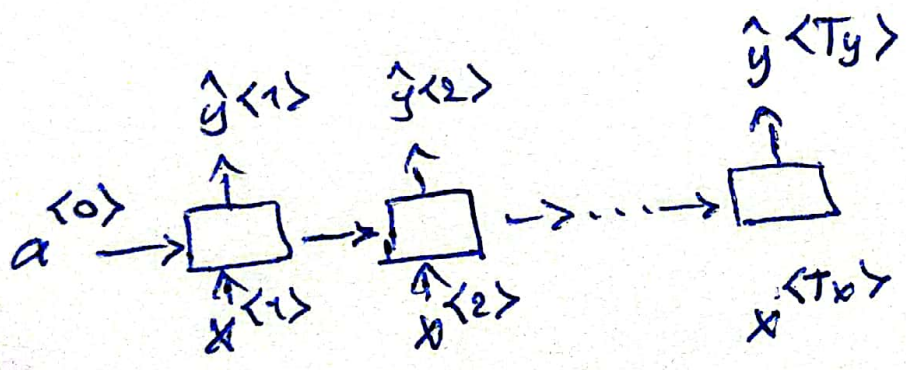


① one-to-one

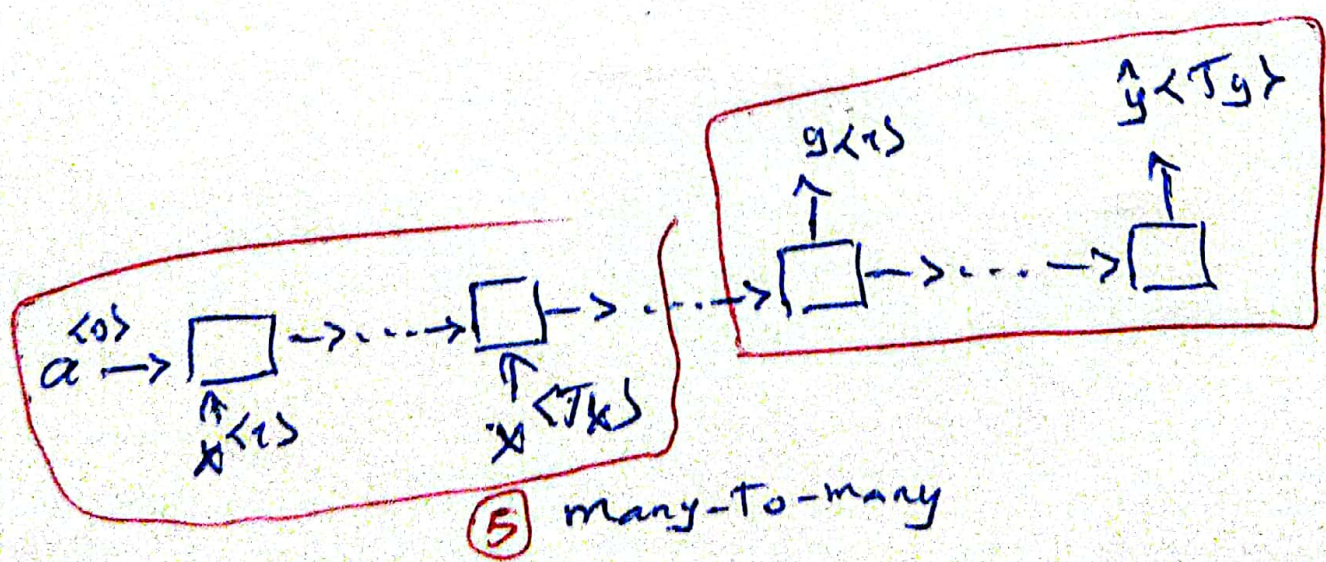
② one-to-many



③ many-to-one



④ many-to-many



⑤ many-to-many

Language model and Sequence generation

exps

speech recognition

The apple and Pair salad.

The apple and Pear salad.

$$P(\text{The apple and } \underline{\text{Pair}} \text{ salad}) = 3.2 \times 10^{-13}$$

$$P(\text{The apple and } \underline{\text{Pear}} \text{ salad}) = 5.7 \times 10^{-10}$$

↓
more possible

$$P(\text{sentence}) = ?$$

$$P(y^{(1)}, y^{(2)}, \dots, y^{(T_y)}) = ?$$

↓
what is the probability of each sentence to be
represented ~~in the text~~ as the next sentence
in ~~the~~ text?
a

How to build a language model?

(11)

Training Set: large corpus of english text.

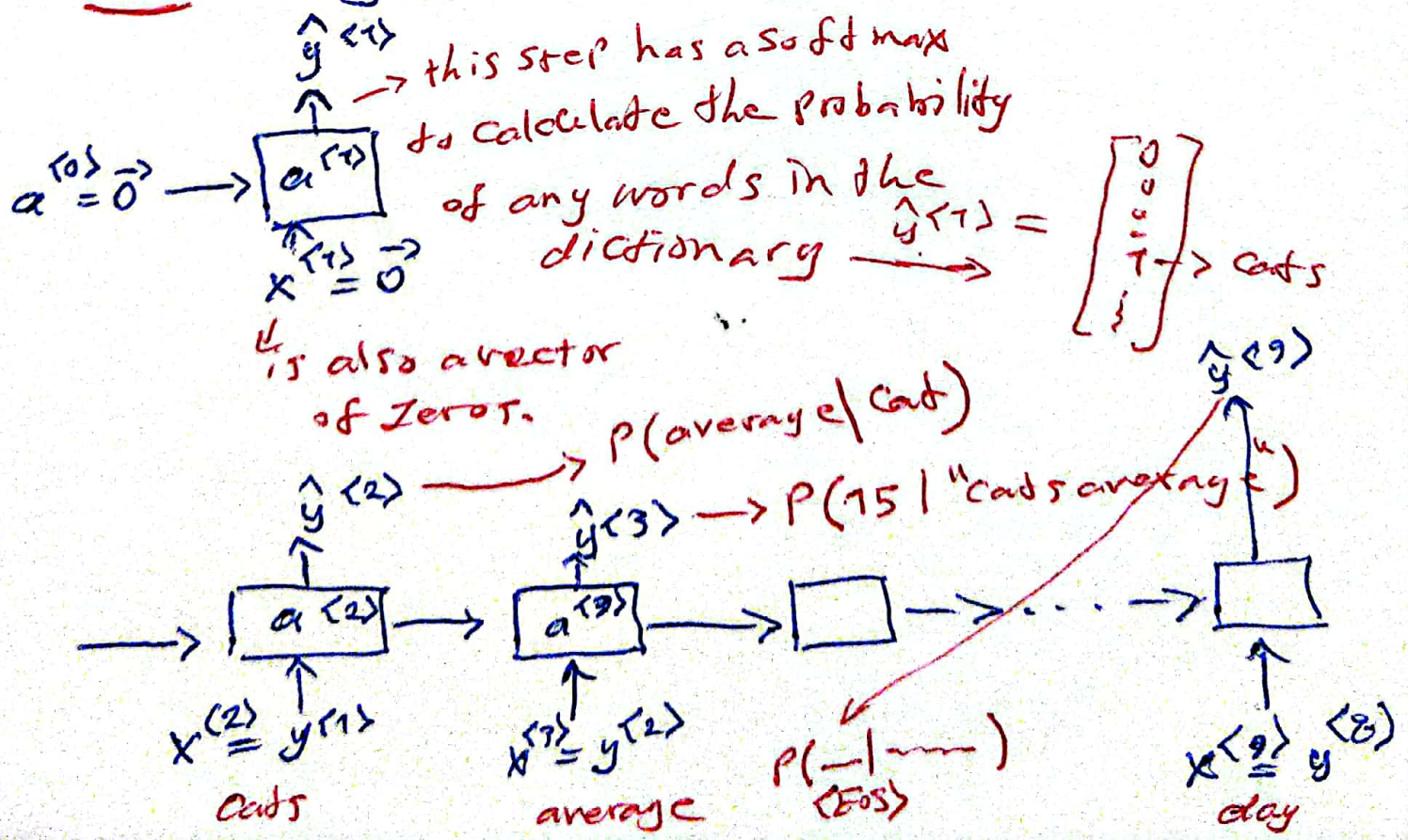


Tokenize: map each sentence in the ~~text~~ training set to a one-hot vector

<EOS>: add this at the end of sentence token to the tokenized training set.

<UNK>: unknown words in vocabulary.

→ Cats average 15 hours of sleep a day. <EOS>



~~after forker~~

(12)

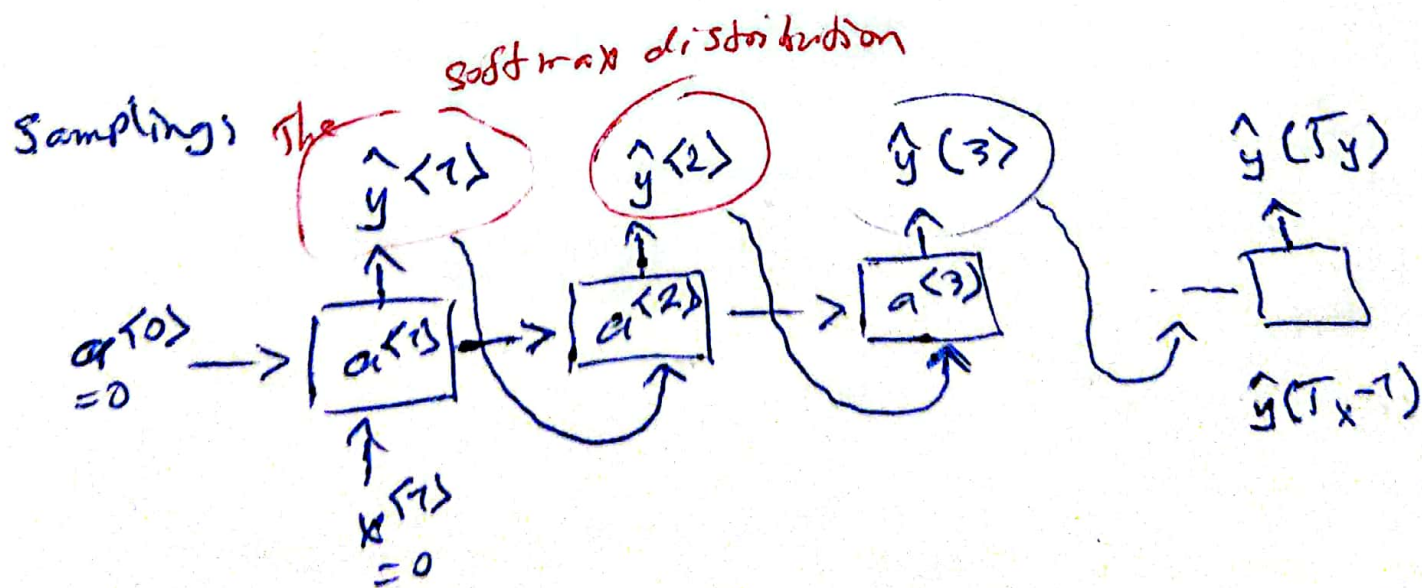
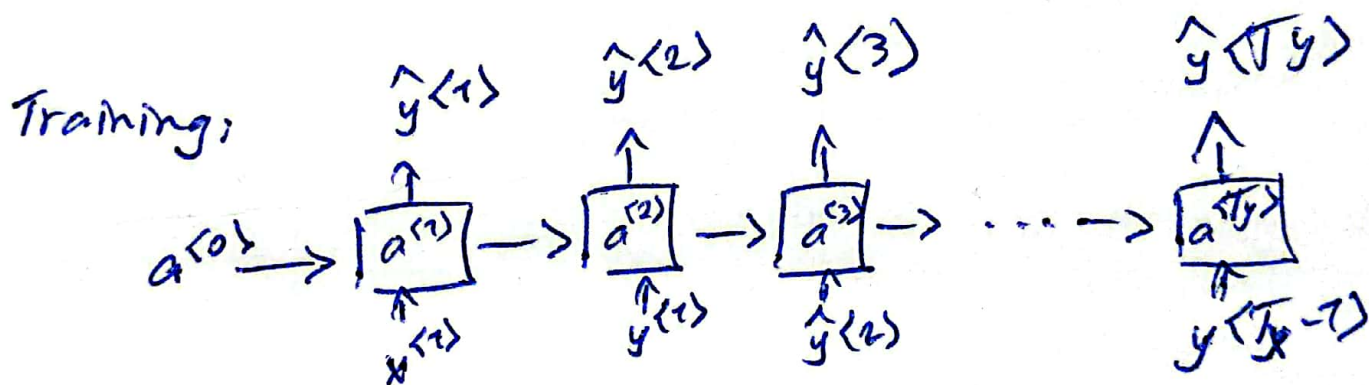
cost function:

$$L(\hat{y}^{(t)}, y^{(t)}) = - \sum_i y_i^{(t)} \log \hat{y}_i^{(t)} \leftarrow \text{loss}$$

$$L = \sum_t L^{(t)}(\hat{y}^{(t)}, y^{(t)})$$

$\leftarrow$ overall loss

Sampling novel sequence. (from a trained RNN)



$\rightarrow p(a) p(a_{(1)}) \dots p(a_{(T_x)}) p(\langle a_{(T_x)} \rangle)$ *RP*
RP: random choice

character-level language model

vocabulary = [a, c, e, r, o, n, ., -, Z, d, e, <UNK>]

vocabulary = [a, b, c, ..., Z, $\backslash$, ., , ; , 0, 1, ..., 9, A, B, -]

$y^{(1)} y^{(2)} y^{(3)}$

in a character level vocabulary $y^{(1)}, y^{(2)}, \dots$ will be
character not words.

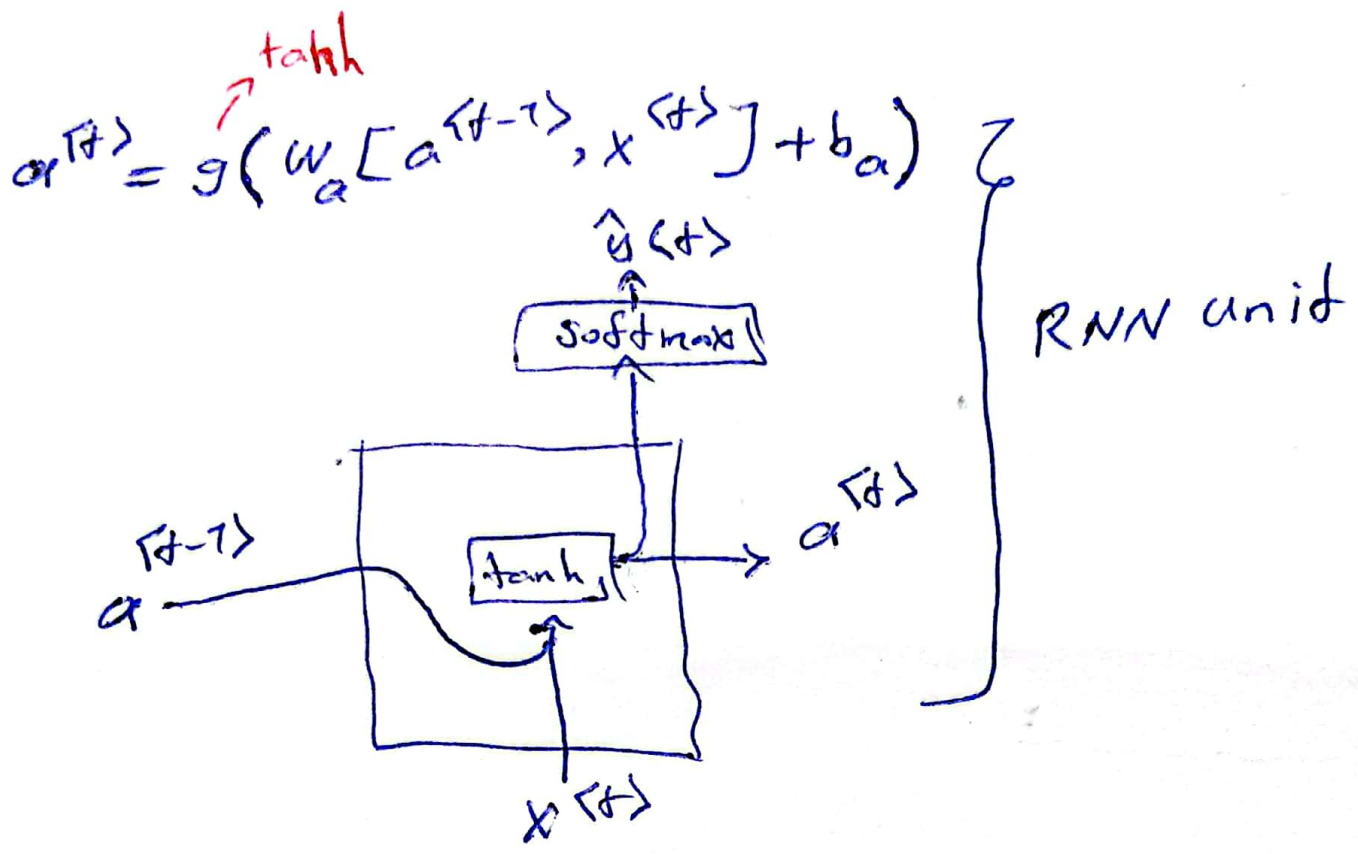
↓
much more computationally intense
compare to word-level language model

vanishing gradient problem in RNNs

- for gradient exploding: gradient clipping is a
solution

- for gradient vanishing: ρ

- Gated Recurrent unit (GRU)



c = memory cell

$$c^{(t)} = a^{(t)}$$

$$\tilde{c}^{(t)} = \tanh(w_c [c^{(t-1)}, x^{(t)}] + b_c)$$

output is 0 or one (sigmoid activation function)

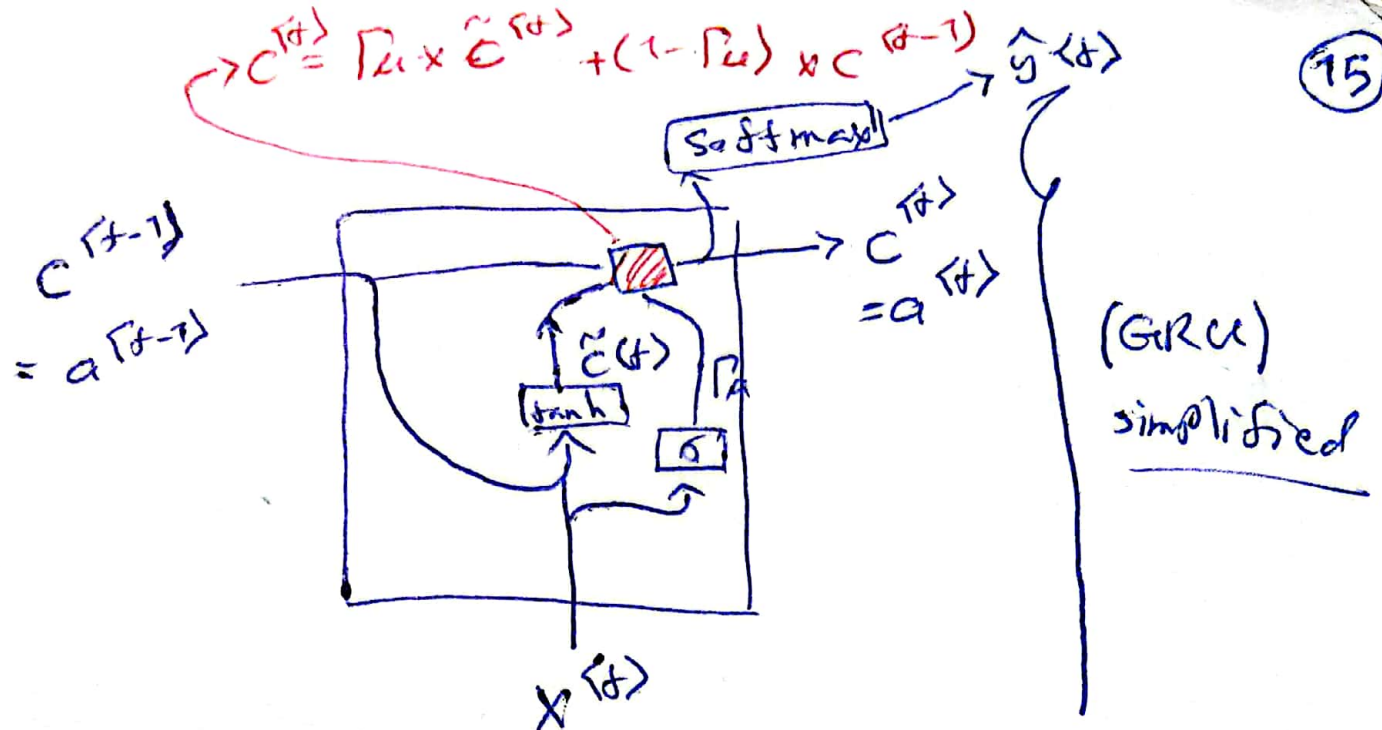
$$\Gamma_u = \sigma(w_u [c^{(t-1)}, x^{(t)}] + b_u)$$

gate $\nearrow$ "update"

$$c^{(t)} = \Gamma_u \times \tilde{c}^{(t)} + (1 - \Gamma_u) \times c^{(t-1)}$$

The cell, which already old ... , was full.

$\rightarrow c^{(t)} = ?$ $\rightarrow ?$
 $\Gamma_u \rightarrow$ when to update $c^{(t)}$?



Full GRU:

$\Gamma_r \times$

$$\tilde{c}^{(t)} = \tanh(W_c [c^{(t-1)}, x^{(t)}] + b_c)$$

$$\Gamma_u = \sigma(W_u [c^{(t-1)}, x^{(t)}] + b_u)$$

$$c^{(t)} = \Gamma_u \times \tilde{c}^{(t)} + (1 - \Gamma_u) \times c^{(t-1)}$$

$\Gamma_r = \sigma(W_r [c^{(t-1)}, x^{(t)}] + b_r)$

Long short Term memory (LSTM):

$$\tilde{C}^{(t)} = \tanh(w_c [a^{(t-1)}, x^{(t)}] + b_c)$$

$$\Gamma_u = \sigma(w_u [a^{(t-1)}, x^{(t)}] + b_u) \rightarrow \text{(update)}$$

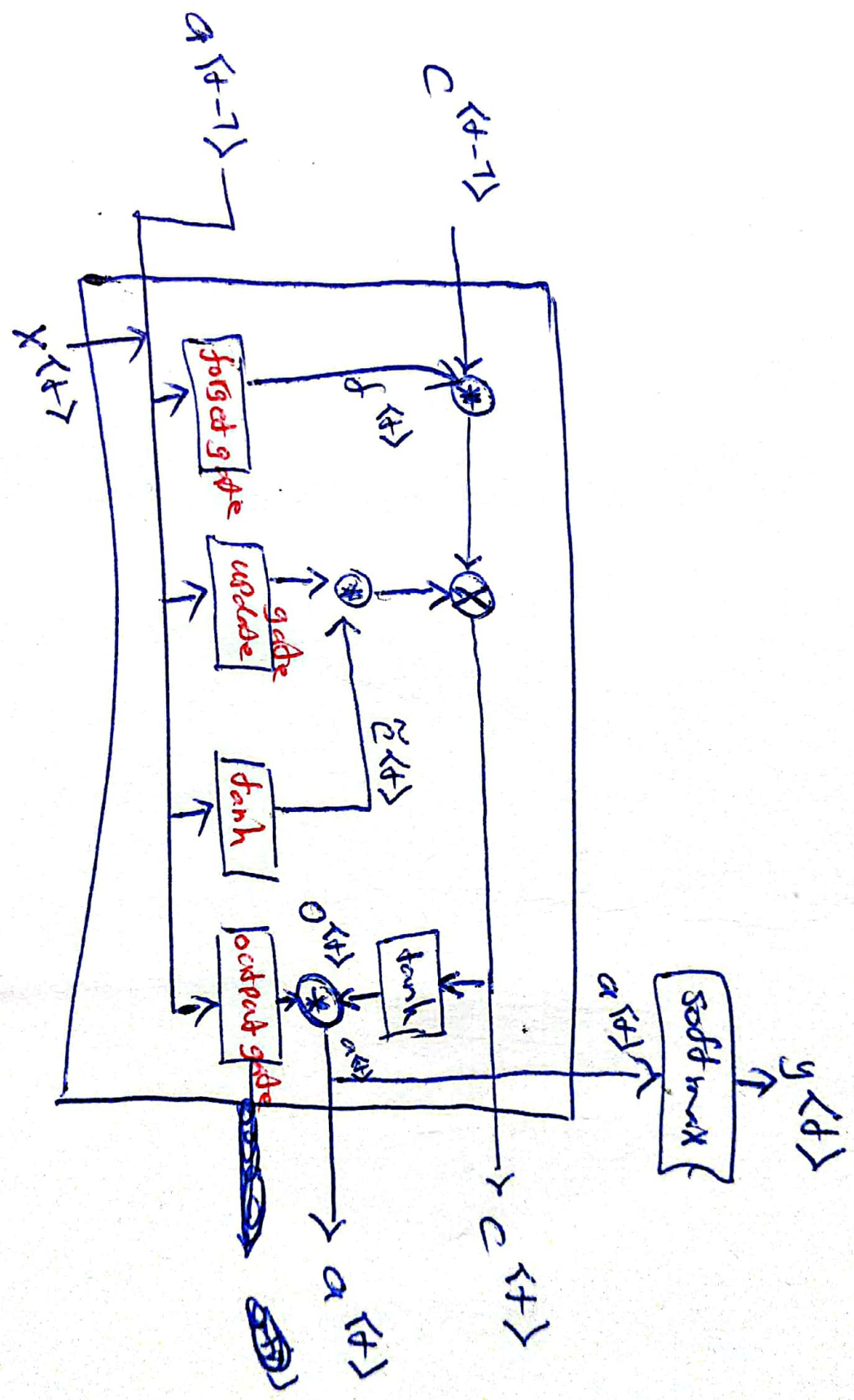
$$\Gamma_f = \sigma(w_f [a^{(t-1)}, x^{(t)}] + b_f) \rightarrow \text{(forget)}$$

$$\Gamma_o = \sigma(w_o [a^{(t-1)}, x^{(t)}] + b_o) \rightarrow \text{(output)}$$

$$C^{(t)} = \Gamma_u \times \tilde{C}^{(t)} + \Gamma_f \times C^{(t-1)}$$

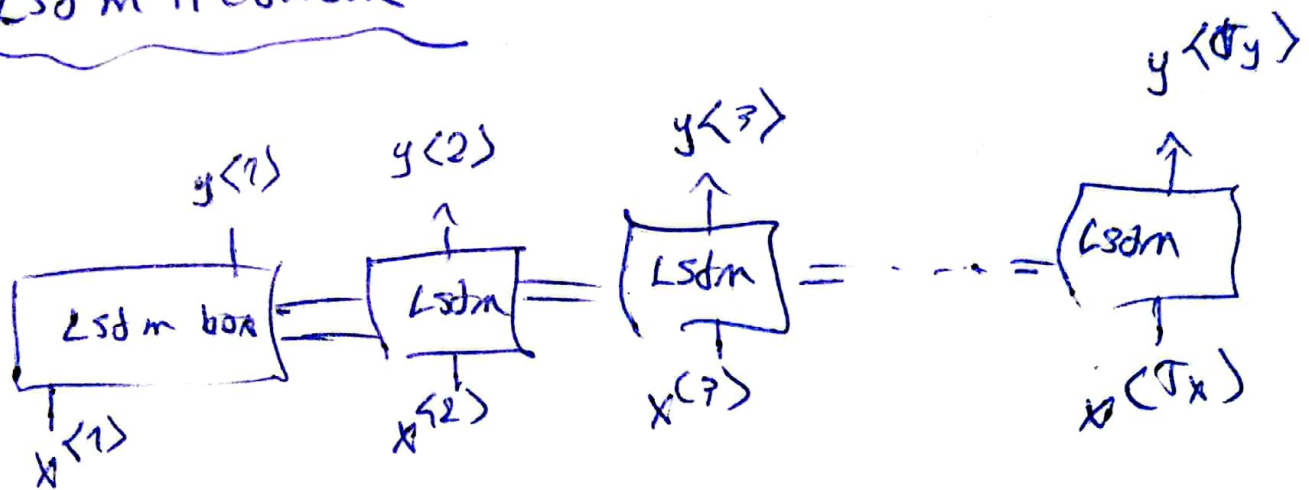
~~$$a^{(t)} = \Gamma_o \times \tanh(C^{(t)})$$~~

$$a^{(t)} = \Gamma_o \times \tanh(C^{(t)})$$



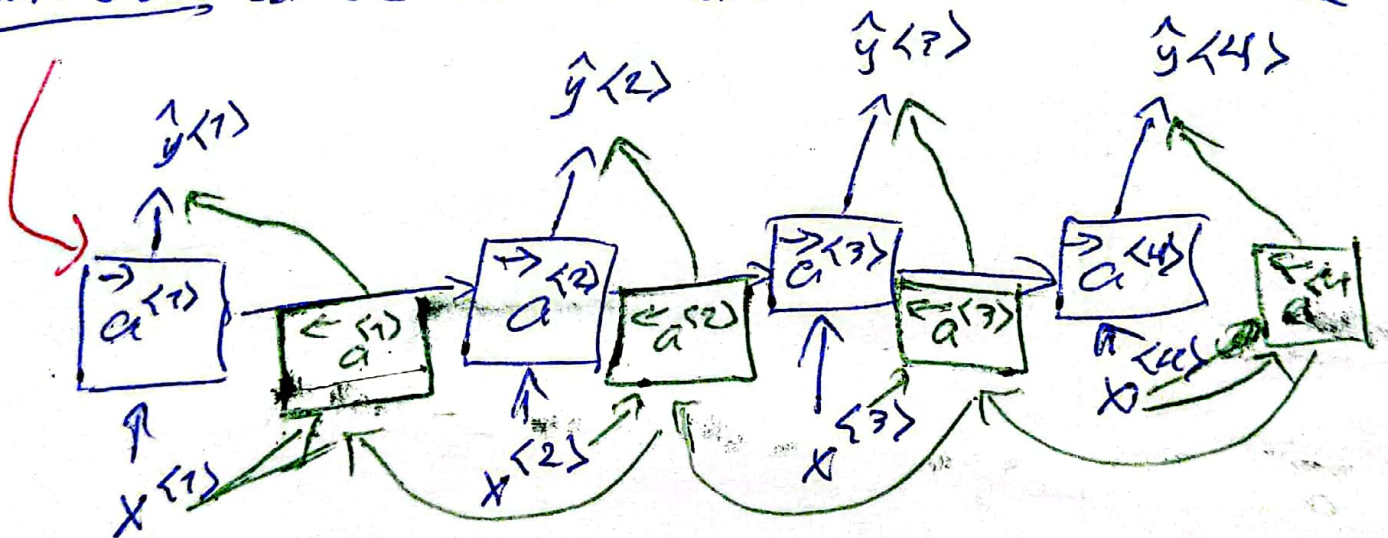
(LSTM structure)

Lstm network



Bidirectional RNN (BRNN)

each block can be Lstm or GRU or ~~simple~~ classical RNN block



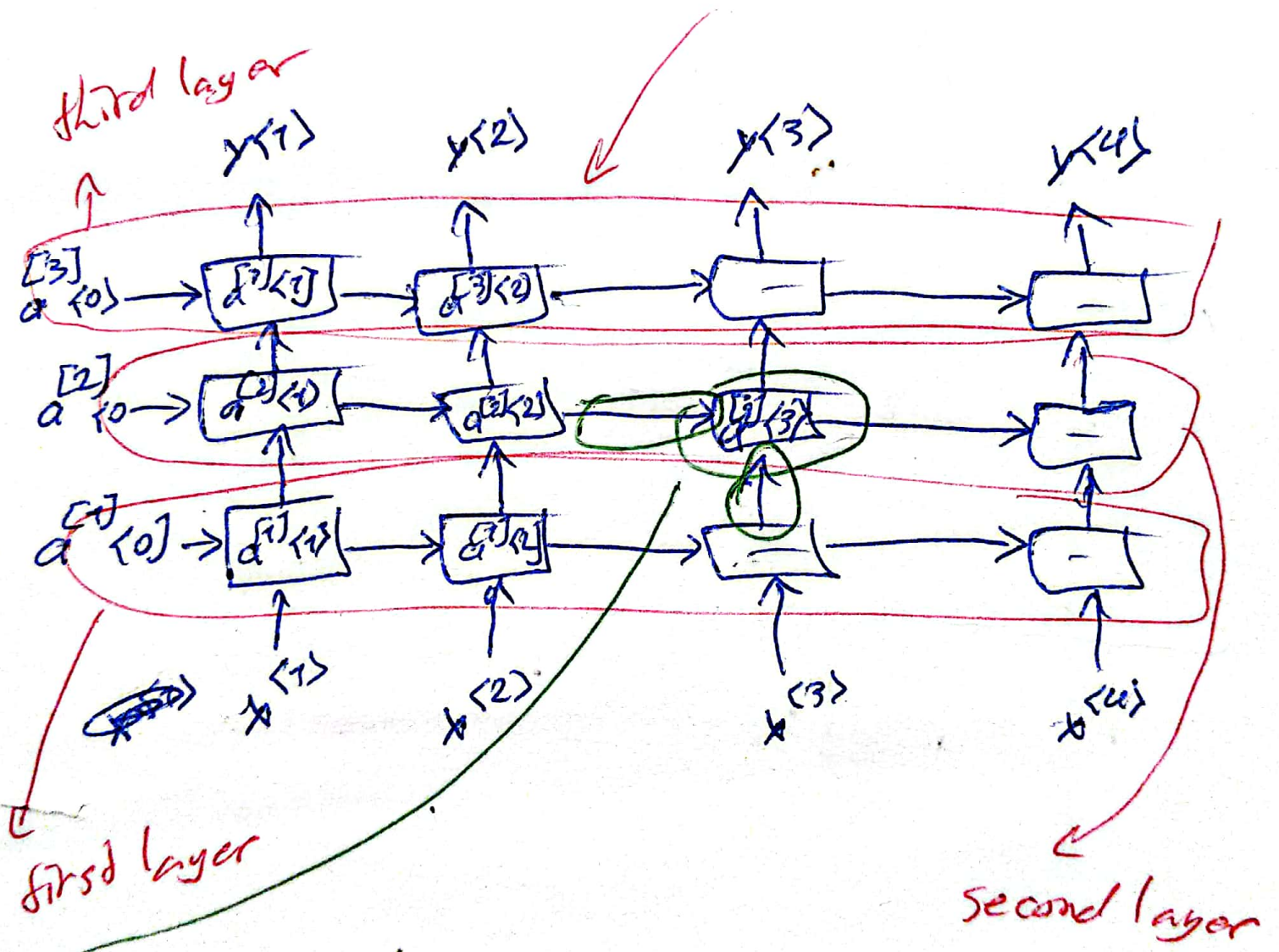
$$\hat{y}^{(t)} = g(w_y [\vec{a}^{(t)}, \overleftarrow{a}^{(t)}] + b_y)$$

exp, "He said Teddy Roosevelt ..."

$\hat{y}^{(t)}$ ← to predict $y^{(t)}$, it uses info from both side of the sentence

Deep RNNs

a deep RNN with three layers



$$a^{[2]}(3) = g\left(y_a^{[2]} \left[a^{[1]}(2), a^{[1]}(3) \right] + b_a^{[2]}\right)$$

each block can be, RNN, ~~ReLU~~ GRU, LSTM or BRNN.

Introduction to word Embeddings

20

word representation:

$V = [a, aaron, \dots, zulu, \langle UNK \rangle]$ $|V| = 10,000$ exp
↑

→ index in vocabulary(V)

Man (5393)



man(5393)

→ vector naming



↓
one-hot representation

woman



→ 9853

- How to predict next word in a sentence?

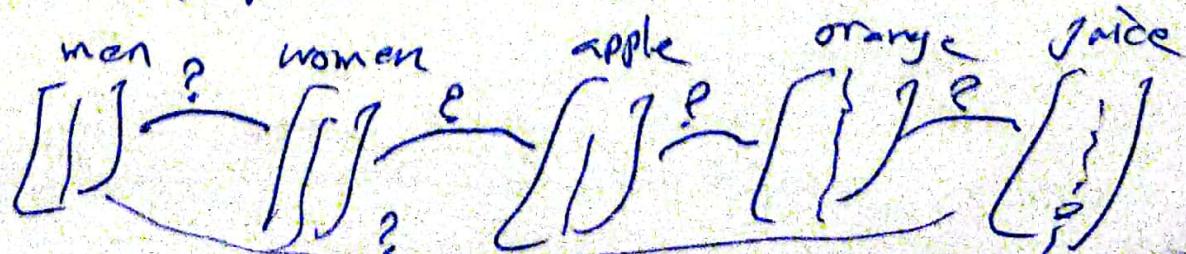
exp,

- I want a glass of orange ?

- I want a glass of apple ?

~~without~~

- without knowing the relationship between words, it is impossible to predict the next word in a sequence



Featurized representation: word embedding

(27)

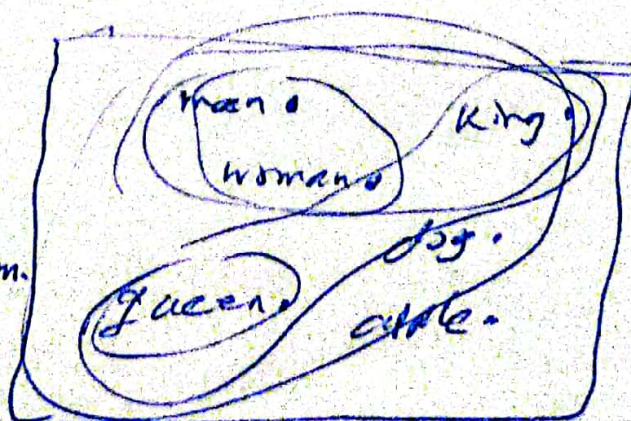
features)	man (5391)	woman (9853)	king (4914)	queen (7157)	apple (496)
Gender	7	1	-0.97	0.97	0.07
Royal	0.01	0.02	0.93	0.93	-0.07
Age	0.03	0.02	0.7	0.69	0.03
Food	0.04	0.04	0.02	0.07	0.93
Size	---	---	---	---	---
cost	---	---	---	---	---

e_{5391} (man) e_{9853} (woman)

- if we have a list of 300 words, each of the words (man, woman, ...) would be a 300 dimensions vector.

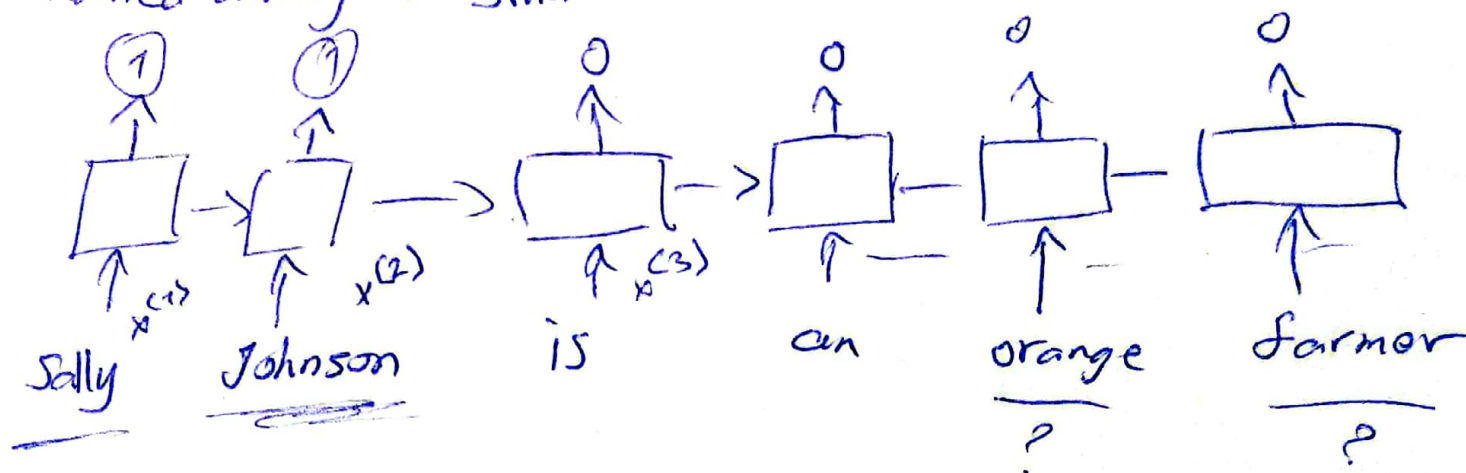
↓ all
 using each of ones (300, 1) vectors ~~data~~ and bring them to a 2D vector for visualization purposes.

Common
 algorithm to do that's
 t-SNE algorithm.



- NLP and word embeddings.

- named entity recognition example;



- usually you can transfer knowledge (weights) from

a model trained on a really big dataset (unlabeled) to

your smaller model which has for example 100K words to be labeled

train on.

1. Learn word embeddings from large text corpora. (1-100B words), (or download pre-trained embedding online.)

2. Transfer embedding to new task with smaller training set. (Sag, 100K words).

3. optional: continue to finetune the word embedding with new data.

Relation between word embedding and face ~~recognition~~ ^{encoding} - (23)

- properties of word embeddings:

- help with analogy reasoning

exp 1

$$e_{\text{man}} - e_{\text{woman}} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{matrix} \text{gender} \\ \text{royal} \\ \text{age} \\ \text{food} \end{matrix}$$

$$e_{\text{king}} - e_{\text{queen}} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

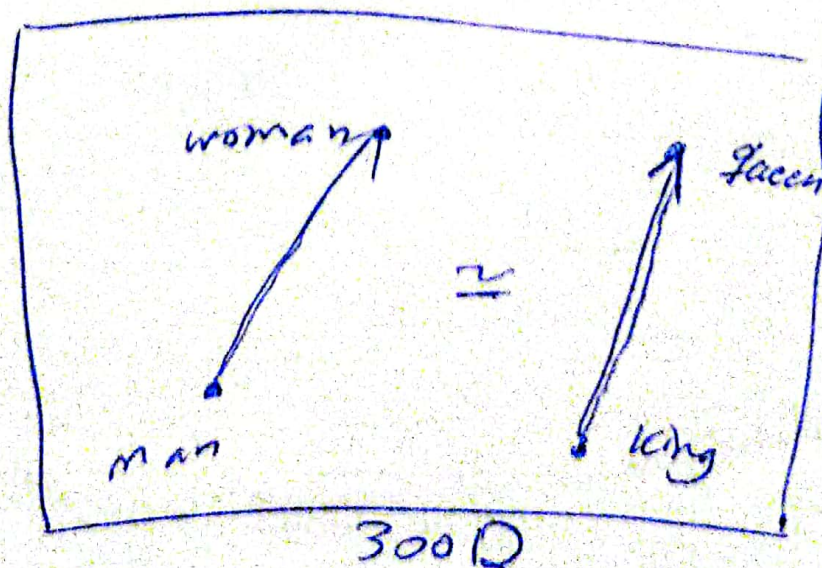
$\approx$

queen

men to woman is same as king to $\textcircled{?}$

$$e_{\text{man}} - e_{\text{woman}} \approx e_{\text{king}} - \textcircled{e_p}$$

(mikolov et al., 2013)



- cosine similarity: most commonly used similarity function.

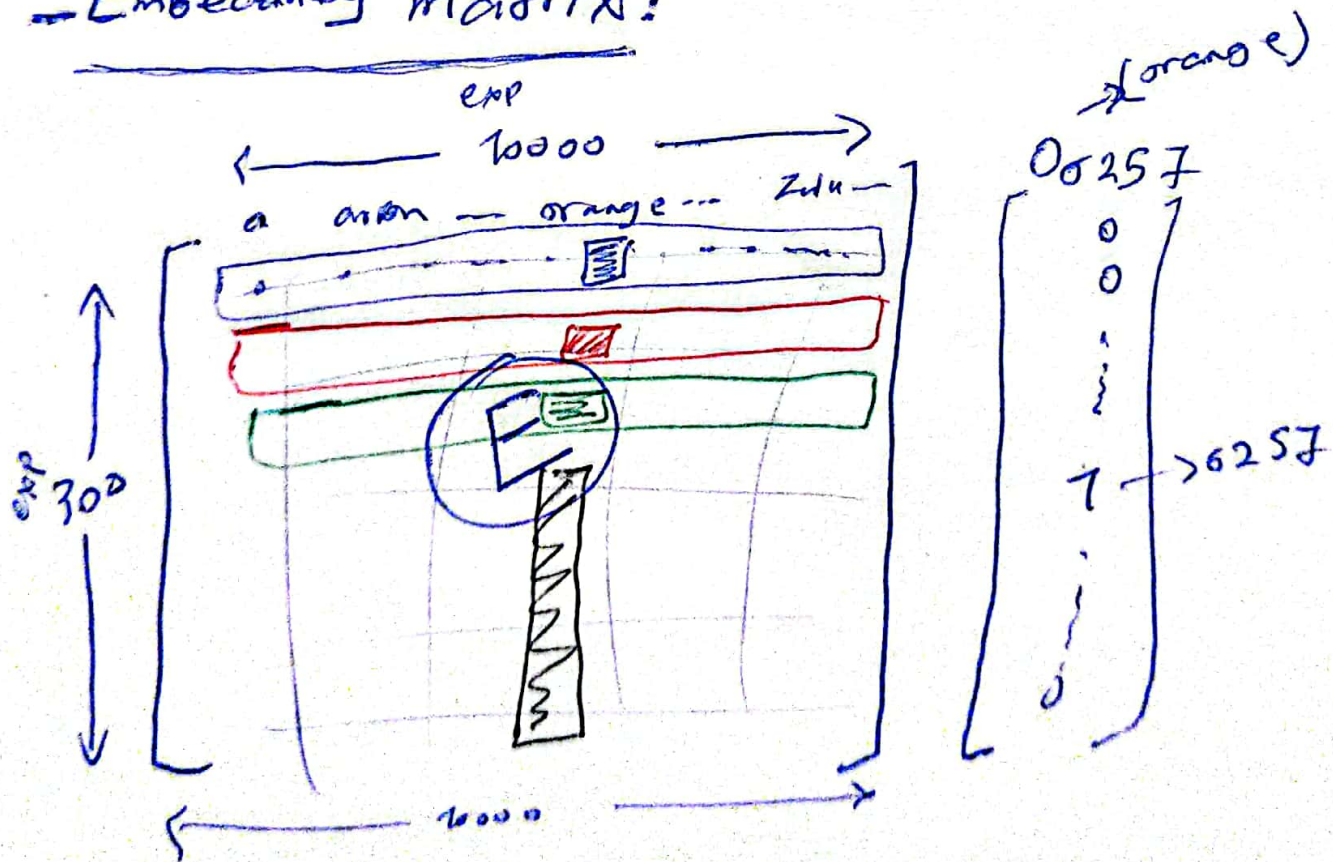
$$\text{sim}(e_w, e_{\text{king}} - e_{\text{man}} + e_{\text{woman}})$$

$$\text{sim}(u, v) = \frac{u^T v}{\|u\|_2 \|v\|_2}$$

1
0
-1

↑ Similarity

- Embedding matrix:



$$E \cdot O = e_{6257}$$

$(300 \times 10000) \cdot (10000, 1)$

$(300, 1)$

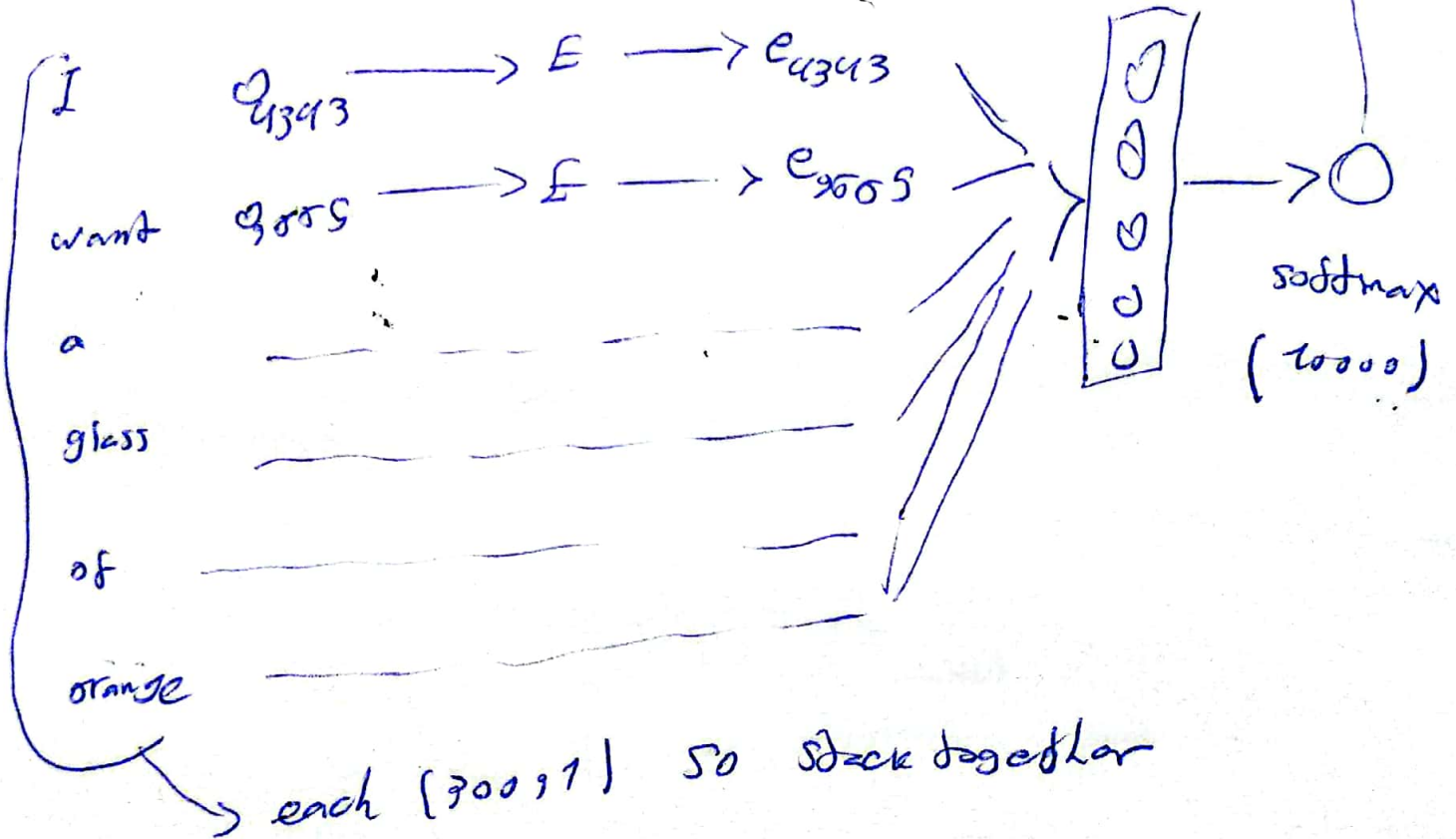
- Neural language models: (Bengio et al, 2003) (25)

exp.

↓ ?

I want a glass of orange

4343 9665 1 7852 6763 6257



Input: (1800, 1)

- other context / target pairs

predict target
feed into a neural network
to learn word embedding

I want a glass of orange juice to go along with my cereal.

↑
target

context, 4 words on left and right.

- Word2Vec algorithm : to learn embedding

- Skip-grams :

I want a glass of orange juice to go along with my cereal.

<u>Context</u>	<u>Target</u>
orange	juice
orange	glass
orange	my ↗
orange	

randomly chosen targets

~~softmax~~

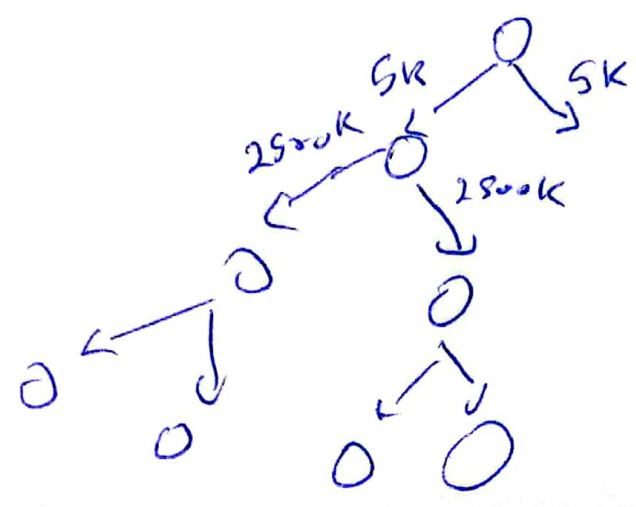
$$\text{softmax: } p(t|c) = \frac{e^{\theta_t^T e_c}}{\sum_{j=1}^{\text{vocab}} e^{\theta_j^T e_c}}$$

* θ_t = parameter associated with word t

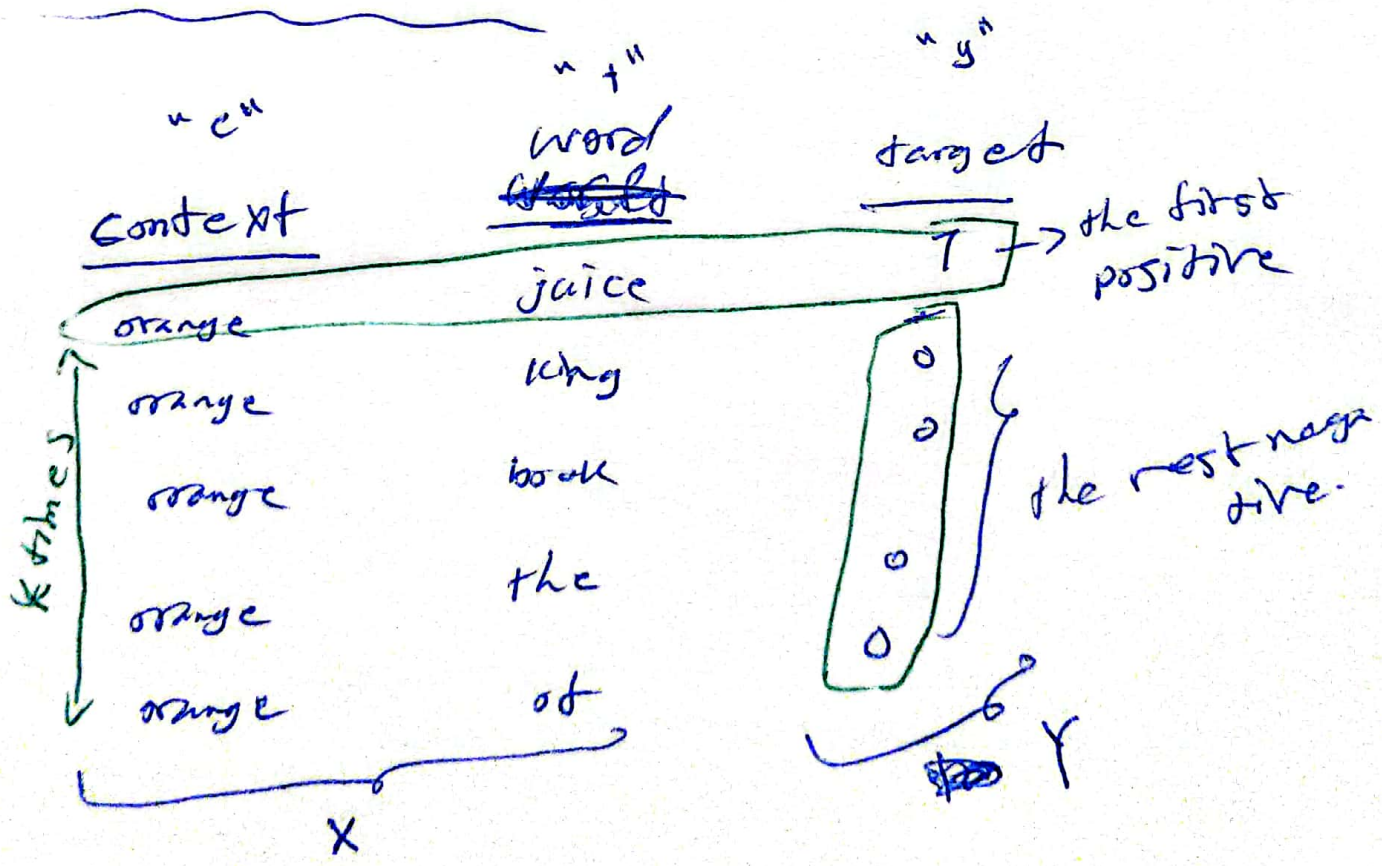
$$L(\hat{y}, y) = - \sum_{i=1} y_i \log \hat{y}_i$$

- problems with softmax classification: (very expensive)

one solution: use hierarchical softmax.



- negative sampling:



a supervised learning algorithm will be trained on the created table

$$P(y=1 | c, t) = \sigma(\theta_t^T e_c)$$

- GloVe algorithm (global vectors for word representation)

Cost

X_{ij} = # times j appears in the context of i .

Diagram illustrating the context and word relationship for X_{ij} :

```

    graph TD
      C1[c] --> i1[i]
      C2[c] --> i2[i]
      i1 --> j[j]
      i2 --> j[j]
  
```

$$\text{minimize} : \sum_{i=1}^{\text{words}} \sum_{j=1}^{\text{words}} f(x_{ij}) (\theta_i^T e_j + b_i + b_j - \log x_{ij})^2$$

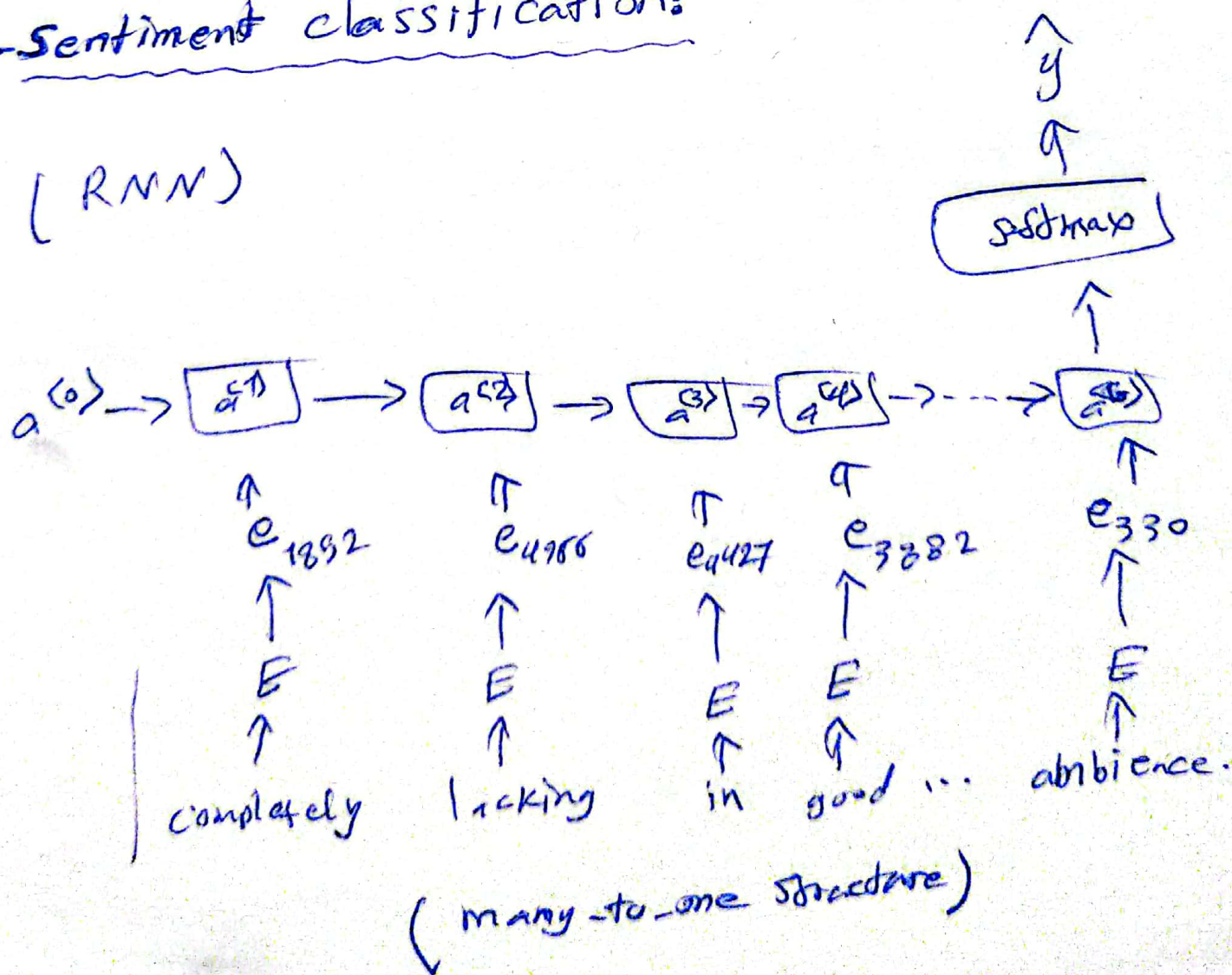
Diagram illustrating the GloVe loss function components:

```

    graph TD
      i[i] --> f[f(x_ij)]
      j[j] --> f[f(x_ij)]
      i --> theta[theta_i^T]
      j --> e[e_j]
      theta --> sum[+]
      b_i[b_i] --> sum
      b_j[b_j] --> sum
      sum --> log[-log x_ij]
      log --> sq[^2]
  
```

- Sentiment classification:

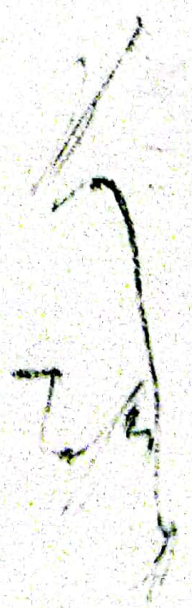
(RNN)



The problem of bias in word embeddings.

Debiasing word embeddings

(Bolukbasi et al., 2016)

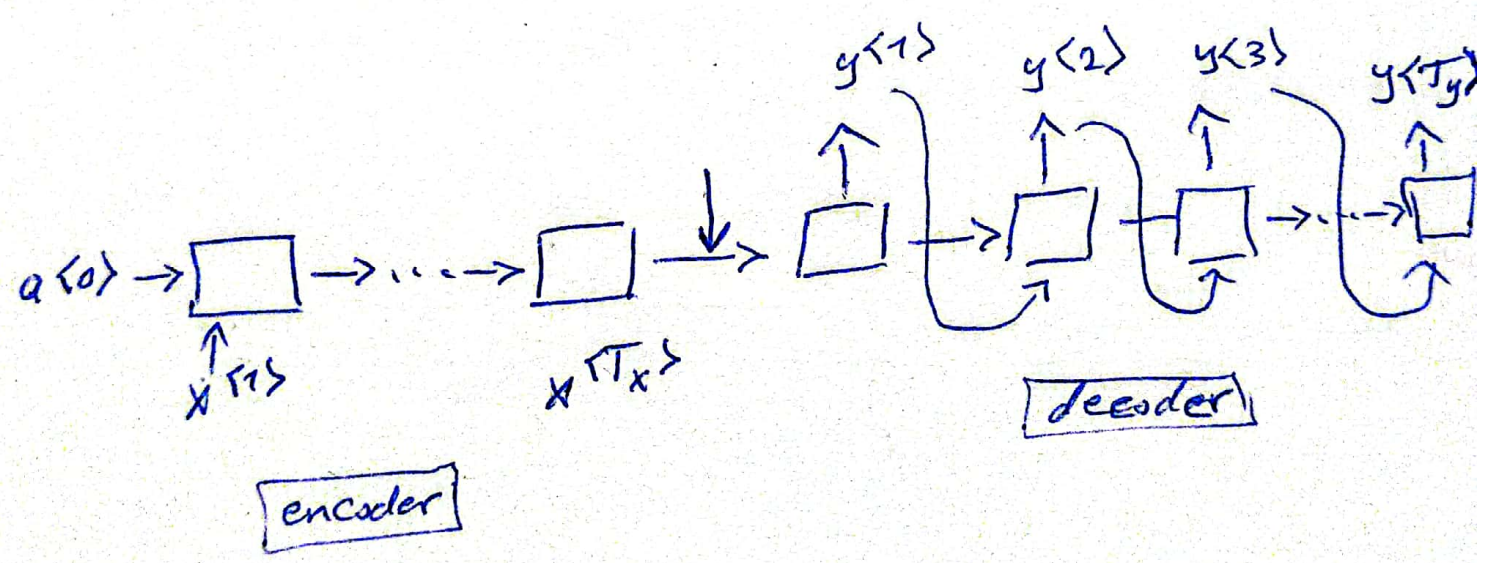


sequence to sequence model 2

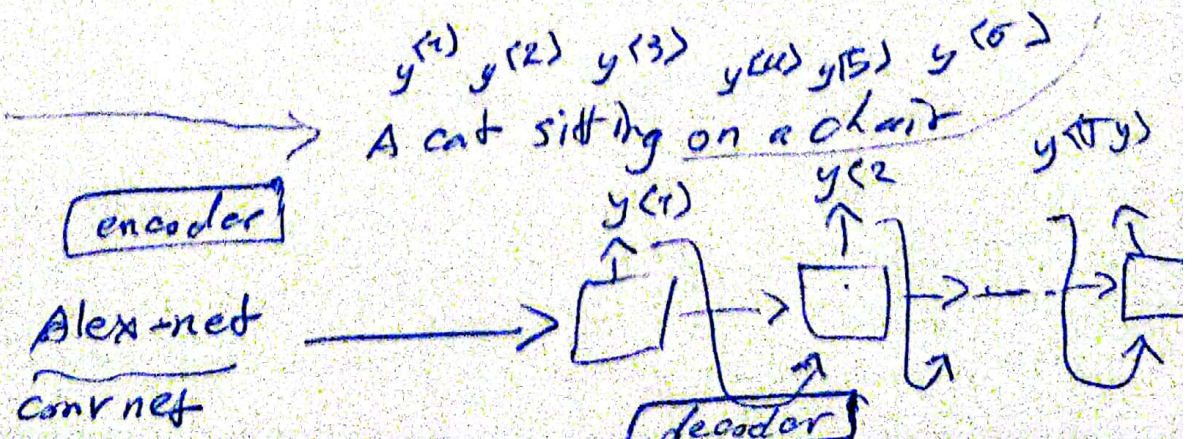
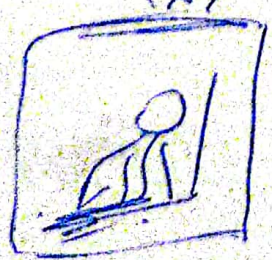
Basic models: machine ~~language~~ translation

exp1: $x^{(1)} \quad x^{(2)} \quad x^{(3)} \quad x^{(4)} \quad x^{(5)}$
 Jane visite l'Afrique en septembre

→ Jane is visiting Africa in september.
 $y^{(1)} \quad y^{(2)} \quad y^{(3)} \quad y^{(4)} \quad y^{(5)} \quad y^{(6)}$

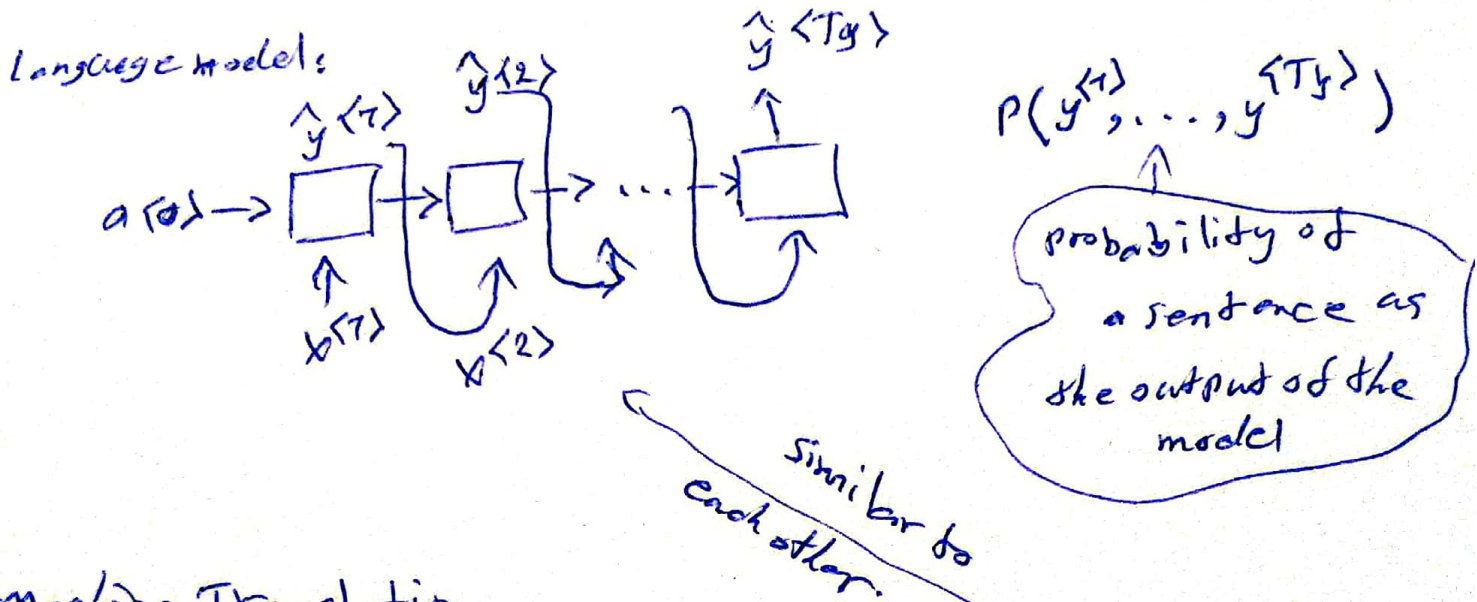


exp2: (image captioning)

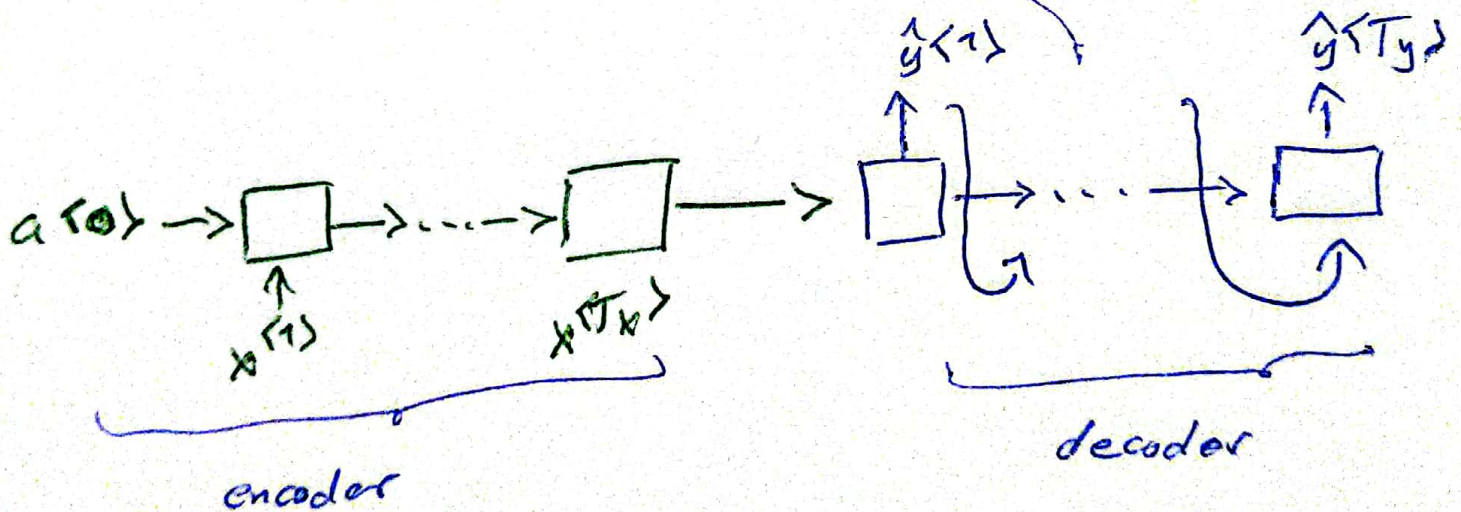


- picking the most likely sentences:

- machine translation as building a conditional language model:



machine Translation:



"conditional language model" $P(y^{(1)}, \dots, y^{(T_y)} | x^{(1)}, \dots, x^{(T_y)})$

finding the most likely translation:

(X) Jane visite l'Afrique en septembre. $P(y^{(1)}, \dots, y^{(T_y)} | x)$ france
 (Y) Jane is visiting Africa in September. english

$$\text{argmax } P(y^{(1)}, \dots, y^{(T_y)} | X)$$

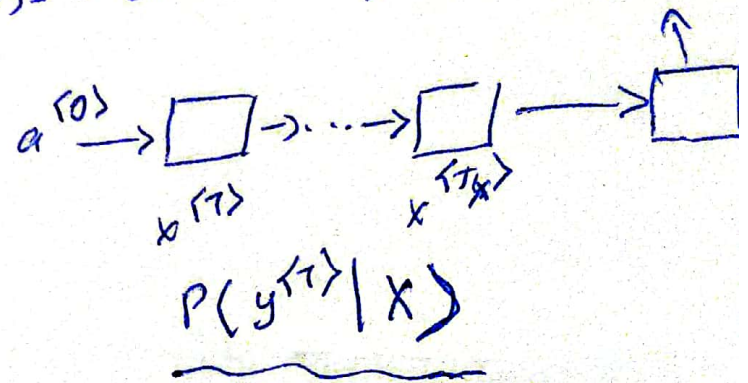
↓
a common algorithm to come up with the best translate is "beam search".

why not a greedy search? ???

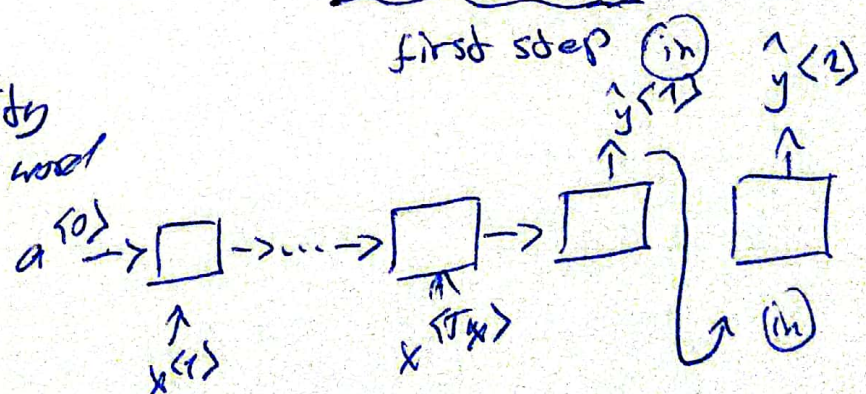
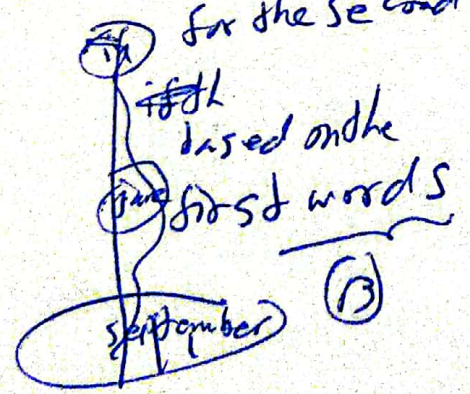
"Beam Search Algorithm"

exp: $\beta = 3$ (beam width) means algorithm try to find the best three phrases as the first $y^{(1)}$

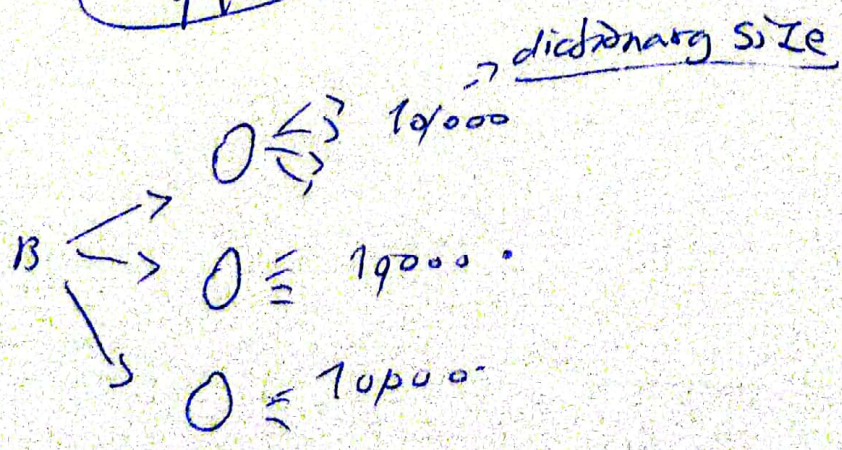
1. The first step is to find the best word (highest probability) as the first word of the sentence:



2. compute the probability for the second word



$$P(y^{(1)}, y^{(2)} | X) = P(y^{(1)} | X) \cdot P(y^{(2)} | X, y^{(1)})$$



3.

$$\left. \begin{aligned} P(y^{(3)} | X, \text{"in september"}) \\ P(y^{(3)} | X, \text{"jane is"}) \\ P(y^{(3)} | X, \text{"jane visits"}) \end{aligned} \right\} B=3$$

In each step one word will be added to each beam.

in this case we used a B (beam) of 3, if we use a Beam of 1 the algorithm is the same as greedy algorithm.

~~reinforcement~~ refinements to Beam search:

- length normalization

$$\arg \max_y \prod_{t=1}^{T_y} P(y^{(t)} | X, y^{(1)}, \dots, y^{(t-1)})$$

if the sentence length is too long, so it results in zero.

so:

$$\arg \max_y \sum_{t=1}^{T_y} \log P(y^{(t)} | X, y^{(1)}, \dots, y^{(t-1)})$$

using log ~~instead of max~~ of probabilities.

$$\arg \max_{y^{(t)}} \left(\frac{1}{T_y} \sum_{t=1}^{T_y} \log P(y^{(t)} | x, y^{(1)}, \dots, y^{(t-1)}) \right)$$

dividing by the
number of words
in the sentence

$$\alpha = 0.7$$

- Beam width B ? computationally
 large B : better result, slower
 small B : worse result, faster

- unlike exact search algorithms like BFS (Breadth First Search) or DFS (Depth First Search), Beam Search runs faster but is not guaranteed to find exact maximum for $\arg \max_{(y)} P(y|x)$.

Error analysis on beam search

exp 1

Jane visite l'Afrique en septembre.

Human: Jane visits Africa in September. (✓)

Algorithm: Jane visited Africa last September. (✗)

RNN (encoder and decoder network)
 Beam Search → which one should be blamed for the bad result?

- RMN computes $\begin{cases} P(y^* | x) \\ P(\hat{y} | x) \end{cases}$

(35)

case 1: $P(y^* | x) > P(\hat{y} | x)$

Beam search chose $\hat{y}$. But y^* attains higher $P(y | x)$.

conclusion: Beam search is at fault.

case 2: $P(y^* | x) \leq P(\hat{y} | x)$

y^* is a better translation than $\hat{y}$. But RMN predicted

$P(y^* | x) \leq P(\hat{y} | x)$.



conclusion: RMN model is at fault.

- BLEU Score:

- How to evaluate the output of a machine translation, when there are multiple translations for one input sentence?

ex: 1

French: Le chat est sur le tapis.

reference 1: The cat is on the mat.

reference 2: There is a cat on the mat.

when the generated algorithm translate is close to one of the reference, the answer will be evaluate with the highest score.

$$P_1 = \frac{\sum_{\text{unique} \in \hat{y}} \text{Count}_{\text{clip}}(\text{unigram})}{\sum_{\text{unique} \in \hat{y}} \text{Count}(\text{unigram})}$$

unigram

$$P_n = \frac{\sum_{n\text{-gram} \in \hat{y}} \text{Count}_{\text{clip}}(n\text{-gram})}{\sum_{n\text{-gram} \in \hat{y}} \text{Count}_{\text{clip}}(n\text{-gram})}$$

P_n = Blue score on n-grams only

combine Blue score: (P_1, P_2, P_3, P_n)

$$\text{BP. EXP} \left(\frac{1}{4} \sum_{n=1}^4 P_n \right)$$

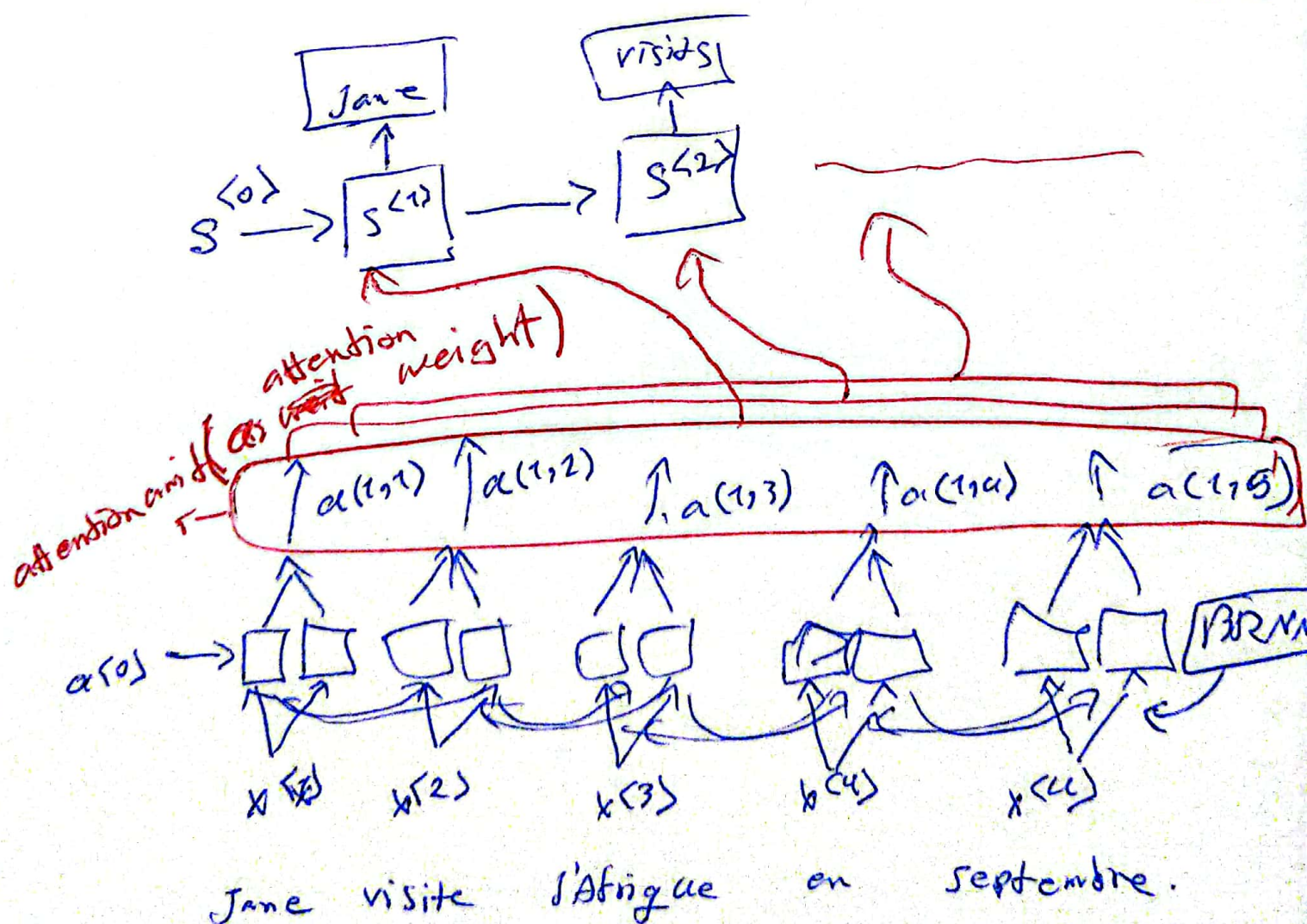
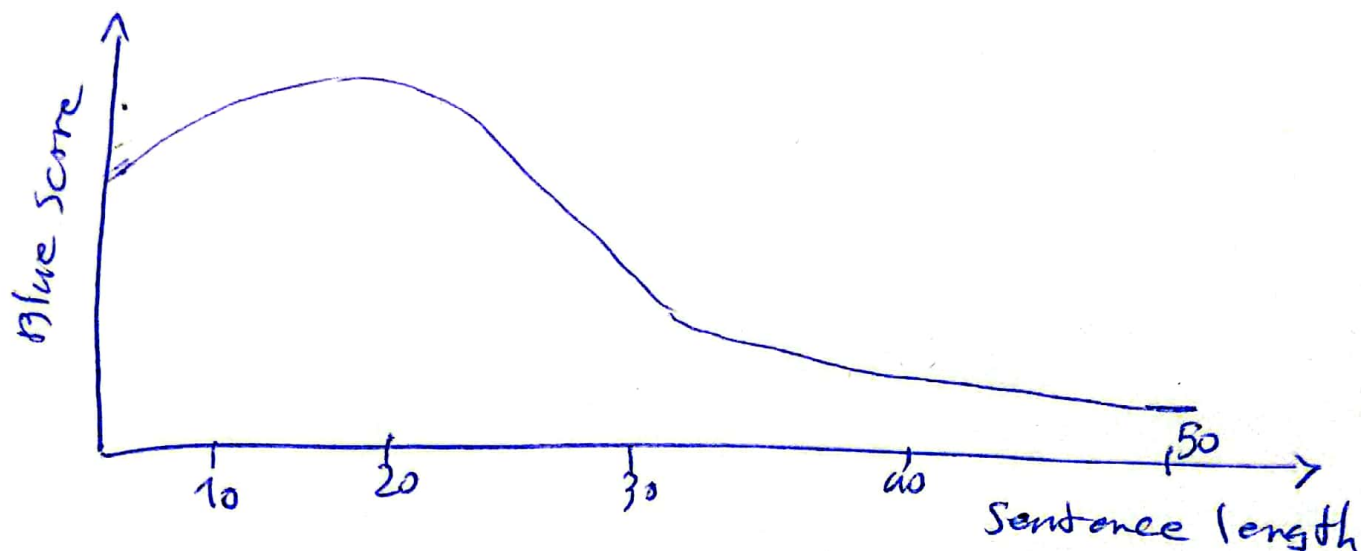
BP = brevity penalty

BP =

$$\begin{cases} 1 & \text{if NT-output-length} > \text{reference-output-length} \\ \exp(1 - \text{reference-output-length} / \text{NT-output-length}) & \text{otherwise} \end{cases}$$

-Attention model intuition:

-The problem of long sequences:

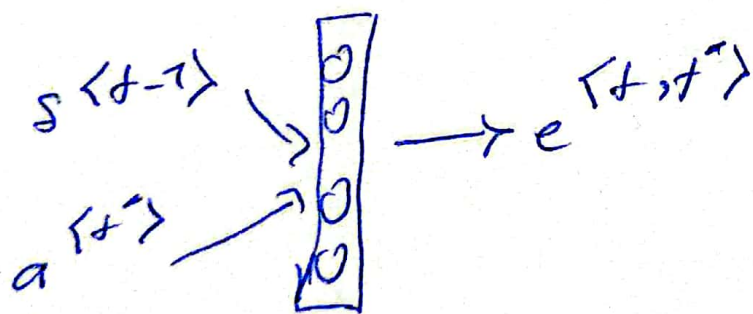


Attention model

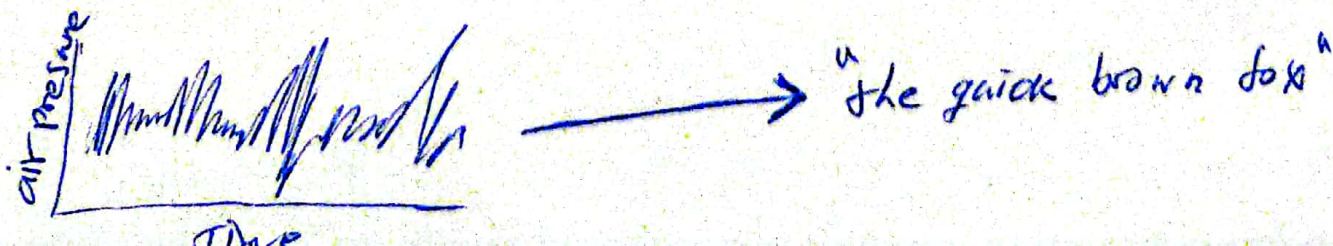
comparing attention

$\alpha_{\langle t, t' \rangle}$ amount of attention $y^{\langle t \rangle}$ should pay to $a^{\langle t' \rangle}$.

$$\alpha_{\langle t, t' \rangle} = \frac{\exp(e^{\langle t, t' \rangle})}{\sum_{t'=1}^T \exp(e^{\langle t, t' \rangle})}$$



Speech recognition



- Trigger word detection

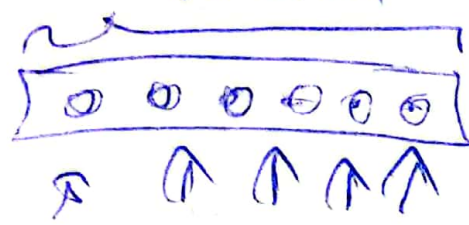
exps

{ Alexa (Amazon Echo)
key siri (Apple siri)
--

- Attention + CNN $\rightarrow A^{(1)} A^{(2)} A^{(3)} \dots A^{(n)}$

- Self-Attention

- Multi-head Attention Parallel



CNN

Self-Attention

$A(q, k, V)$ = attention-based vector representation of a word. $(A^{(1)}, \dots, A^{(5)})$

Calculate for each word

RNN Attention

$$a^{(t,t')} = \frac{\exp(e^{(t,t')})}{\sum_{t'=1}^T \exp(e^{(t,t')})}$$

Self Attention

$$A(q, k, V) = \sum_i \frac{\exp(q \cdot k^{(i)})}{\sum_j \exp(q \cdot k^{(j)})} V^{(i)}$$

How to calculate the attention of $x^{(3)}$ ($A^{(3)}$).

$A^{(3)}$

$x^{(1)}$ $x^{(2)}$ $x^{(3)}$ $x^{(4)}$ $x^{(5)}$
 Jane visite J'adrigae en septembre

- for each word (in this case we are going to calculate self-attention for $x^{(3)}$) there are three values, which should be calculated:

- $q^{(3)}$: query
- $k^{(3)}$: key
- $v^{(3)}$: value

$$A(q, k, v) = \frac{\sum_i \exp(q \cdot k^{(i)})}{\sum_j \exp(q \cdot k^{(j)})} v^{(i)}$$

softmax

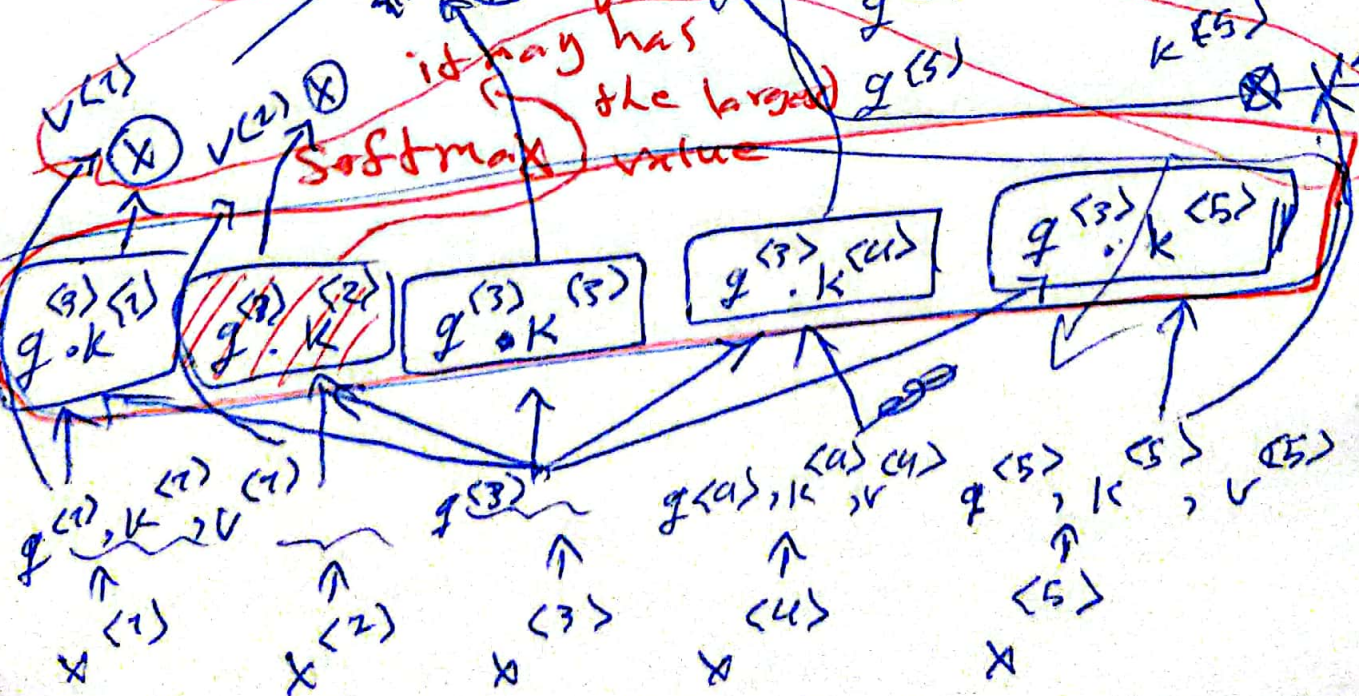
$A^{(3)}$ learnable parameters

$$q^{(3)} = W^q \cdot x^{(3)}$$

$$k^{(3)} = W^k \cdot x^{(3)}$$

$$v^{(3)} = W^v \cdot x^{(3)}$$

Query(Q)	Key(K)	Value(V)
$q^{(1)}$	$k^{(1)}$	$v^{(1)}$
$q^{(2)}$	$k^{(2)}$	$v^{(2)}$
$q^{(3)}$	$k^{(3)}$	$v^{(3)}$
$q^{(4)}$	$k^{(4)}$	$v^{(4)}$
$q^{(5)}$	$k^{(5)}$	$v^{(5)}$



Self-Attention

(41)

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) \cdot V$$

✓ *scaled dot product*

vectorized version of self attention,
which means attention $A^{(1)}, \dots, A^{(S)}$ is
calculated in parallel.

multi-head-Attention: (it is basically a for loop through
~~computed~~ self-attentions.)

in multi-head-Attention the self attention mechanism
will be used multiple times for the same input,
which means there are (W_i) weight matrices
this means each "Head" (self-attention) has its
own W^Q, W^K, W^V matrices.

each head answers a different question ~~about~~
about the current input, so it means the
attention in each head is on a different part of
the sentence.

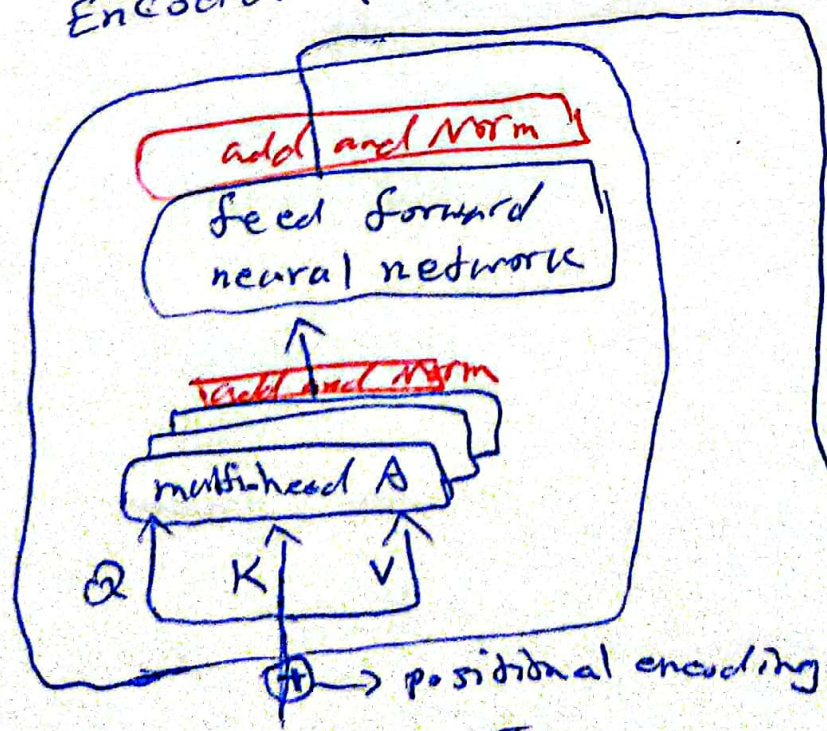
$$\text{multiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) W_o$$

$$\text{head}_i = \text{Attention}(W_i^Q \cdot Q, W_i^K \cdot K, W_i^V \cdot V)$$

- ~~for~~ to make the computation of multihead attention, instead a for loop, all ~~heads~~ ~~com~~ heads' computations can be parallelized, which means it can be calculated at the same time.

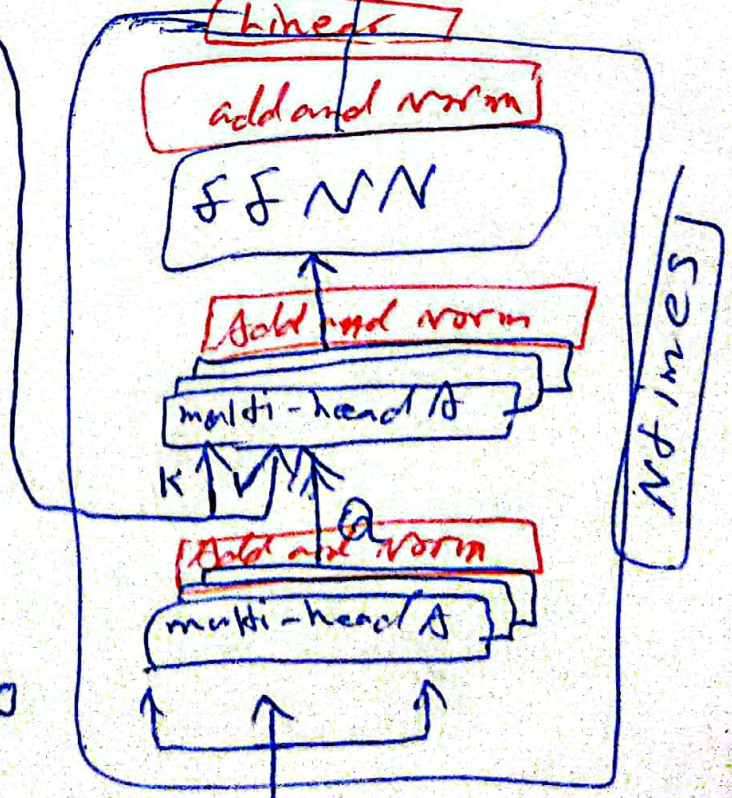
Transformer network

Encoder (N times)
 e.g - 6



$\langle \text{SOS} \rangle x^{(1)}, x^{(2)}, \dots, x^{(T_x)} \langle \text{EOS} \rangle$

Decoder (N times)
 e.g - 6
 softmax
 decoder
 linear



$\langle \text{SOS} \rangle \text{Jane visits Africa in September} \langle \text{EOS} \rangle$

positional encoding;

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000 \frac{2i}{d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000 \frac{2i}{d}}\right)$$